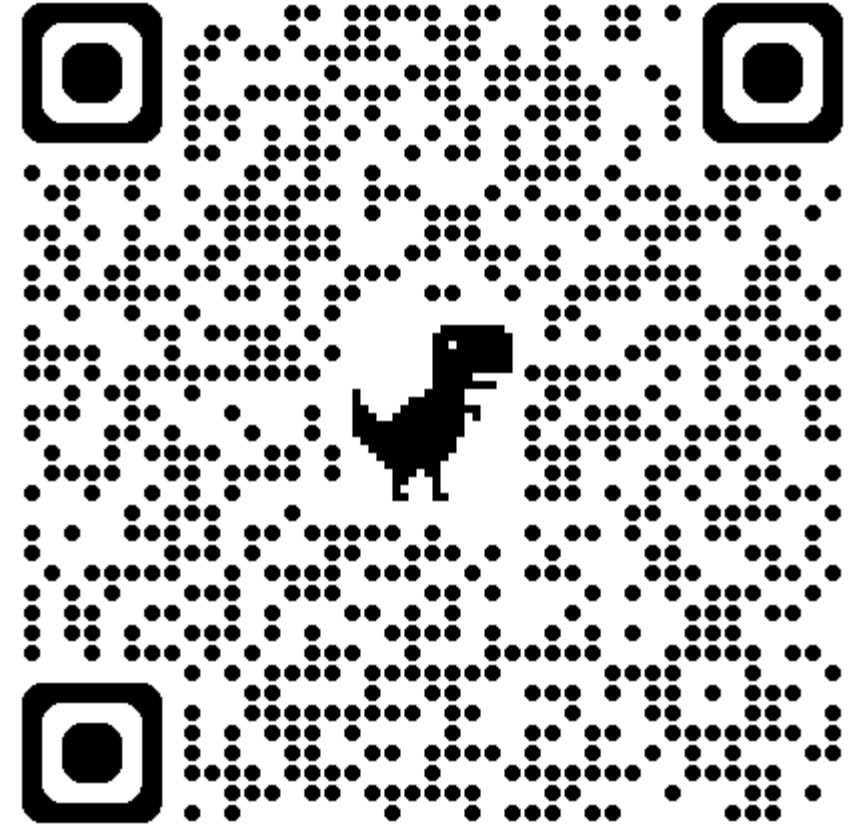


OOP2025F 期中考試公告與考試調查

- 11/06 (星期四) 10:10 – 11:40 筆試 (90 分鐘)
 - 地點：科研大樓 1222 + 1223 電腦教室
- 11/13 (星期四) 09:10 – 12:10 上機考試 (180 分鐘)
 - 地點：科研大樓 1222 + 1223 電腦教室
- https://docs.google.com/forms/d/e/1FAIpQLSdr1-jgBr_K1C2CsOS1s8KDH929Aec9xP3gymFL51aSxM0Ykw/viewform



行程表 (暫定)

W	Date	Lecture	Homework	TA Class
1	09/10, 09/11	Course Information / No class - Registration	Homework 00 (Wed.)	V
2	09/17, 09/18	Git Introduction		
3	09/24, 09/25	OOP Concept	Homework 01 (Wed.)	V
4	10/01, 10/02	Introducing What C++ is		
5	10/08, 10/09	Class / Essential STL Introduction	Homework 02 (Wed.)	
6	10/15, 10/16	Lec03: Encapsulation		V
7	10/22, 10/23	Lec04: Inheritance	Homework 03 (Wed.)	
8	10/29, 10/30	Lec04: Inheritance		V
9	11/05, 11/06	Physical Hand-Written Midterm	Homework 04 (Mon.)	
10	11/12, 11/13	Physical Computer-based Midterm		
11	11/19, 11/20	Lec05: Polymorphism	Homework 05 (Wed.)	V
12	11/26, 12/27	Lec05: Polymorphism		
13	12/03, 12/04	Lec05: Polymorphism - LAB	Homework 06 (Wed.)	V
14	12/10, 12/11	Lec06: Composition & Interface		
15	12/17, 12/18	Lec06: Composition & Interface	Homework 07 (Wed.)	
16	12/24, 12/25	Lec06: Composition & Interface - Lab/ No class		V
17	12/31, 01/01	Physical Hand-Written Final / No class		
18	01/07, 01/08	Lec07: Experience sharing & OOPL - Preparation / Physical Computer-based Final		

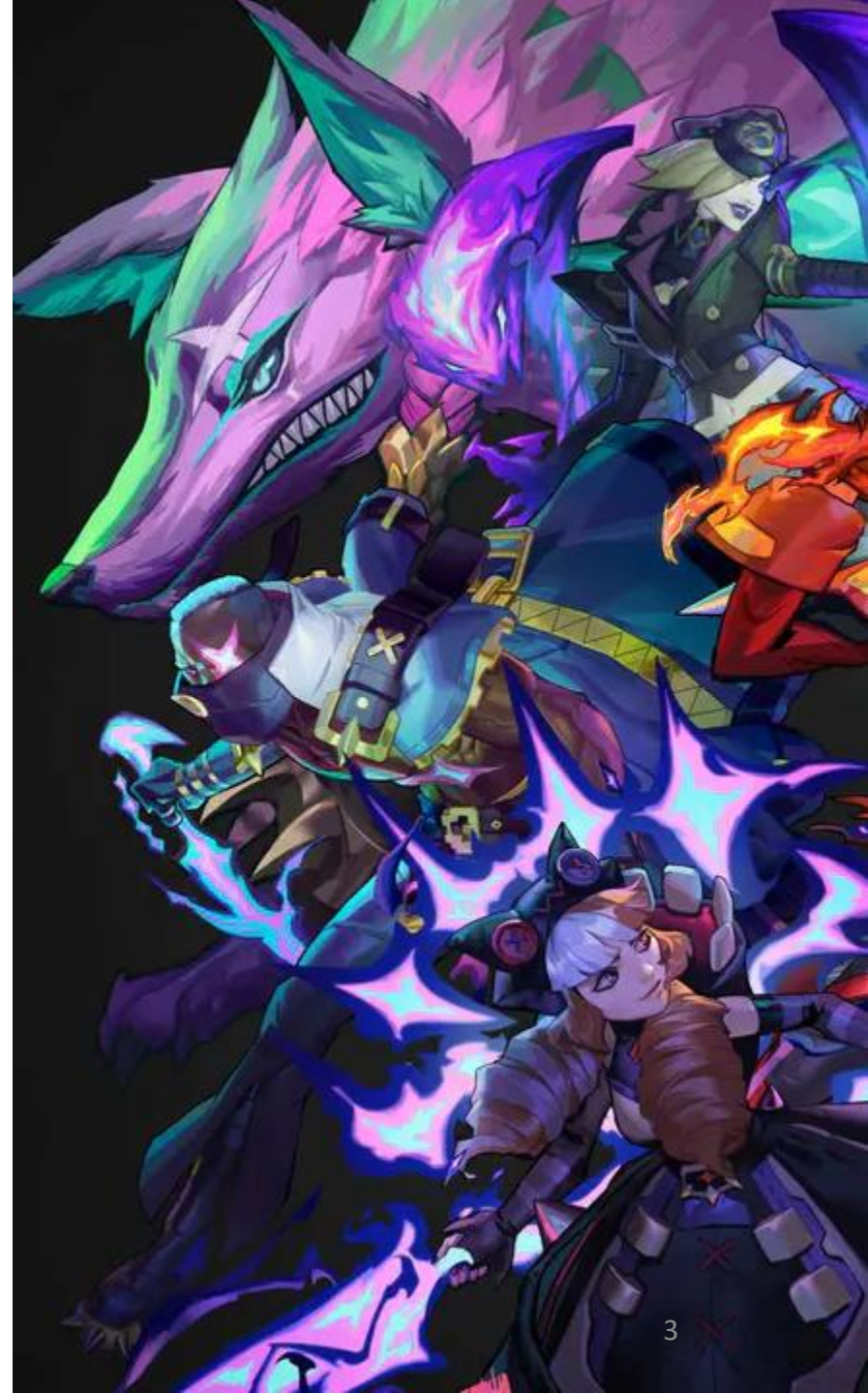
物件導向程式設計

第陸章：繼承

授課講師：孫勤昱博士

國立臺北科技大學 資訊工程系

cysun@ntut.edu.tw



大綱

- 回顧：繼承大概念
- 嘗試解決問題的方向
- 概念：父類別 (Parent) 與子類別 (Children)
- 初始化列表 (Initialize List) 與為何需要初始化列表
- 權限控制：protected
- 繼承與權限控制
- 最終繼承：Final
- 子類：覆寫函數
- 父類：虛擬函數 (Virtual)
- 父類：Final 覆寫 (Override Final)
- 純虛擬函數與虛擬函數

回顧：繼承大概念

- 以 League of Legends 這款遊戲當作例子，我們將再次解釋繼承相關的概念



回顧：繼承大概念

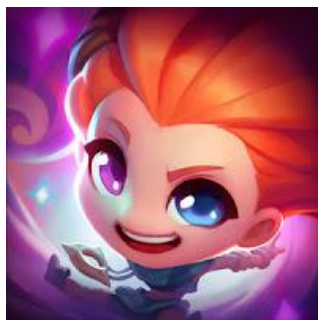
- 作為一個想要寫一個跟 LoL 一樣的遊戲的程式設計師
- 你今天想要「先」撰寫這些角色。



阿璃 Ahri
法師



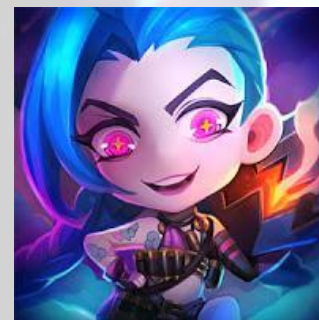
飛斯 Fizz
法師



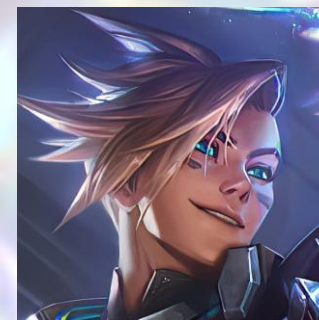
柔伊 Zoe
法師



燼 Jhin
射手



吉因克斯 Jinx
射手



伊澤瑞爾 Ezreal
射手

回顧：繼承大概念

- 針對於這些角色，你先撰寫了阿璃的類別



阿璃 Ahri
法師

Q 鍵：幻玉 (OrbOfDescription)

- 阿璃擲出她的幻玉後再召回，飛出時造成魔法傷害。折返時造成真實傷害

W 鍵：魅火 (FoxFire)

- 阿璃獲得短暫的爆發性跑速加成，並釋放出三團狐火，鎖定附近的敵人進行攻擊

E 鍵：傾城 (Charm)

- 阿璃送出飛吻，傷害並魅惑一名命中的敵方英雄，立刻打斷移動技能，並使對方毫無抵抗的走向她

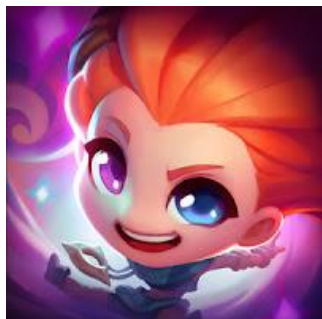
R 鍵：飛仙 (SpiritRush)

- 阿璃向前衝刺並發射魔法彈，傷害附近敵人。飛仙進入冷卻前最多可以施放 3 次，參與敵方英雄的擊殺時，可獲得額外的重新施放次數

```
1  #include <string>
2
3  class Ahri {
4  private:
5      std::string name;
6      int abilityPower;
7      int health;
8  public:
9      /* Getter */
10     std::string GetName();
11     int GetAbilityPower();
12     int GetHealth();
13
14     /* Setter */
15     void SetName();
16     void SetAbilityPower();
17     void SetHealth();
18
19     /* Wizard Only*/
20     void WizardLevelUp();
21
22     /* Ahri-Only Skill */
23     void OrbOfDescription();
24     void FoxFire();
25     void Charm();
26     void SpiritRush();
27 };
```

回顧：繼承大概念

- 然後撰寫了柔伊的類別



柔伊 Zoe
法師

Q 鍵：星迴百轉 (MoreSparkles)

- 柔依發射一顆星星，飛行期間她可以重啟技能，將星星導向至新的位置。星星在直線上飛行的距離越長，造成的傷害越高

W 鍵：法術竊盜 (PaddleStar)

- 柔依可拾起敵方召喚師技能及主動道具效果的碎片，並施放一次該召喚師技能或主動道具效果。每次施放一個召喚師技能，她會獲得 3 顆星彈，其會朝最接近的敵方目標發射

E 鍵：沉睡夢域 (SpellThief)

- 使目標疲倦，接著陷入沉睡。沉睡期間，目標的魔法防禦將會降低。首次對該目標的攻擊會喚醒目標並造成雙倍傷害，此傷害不會超過系統設定的上限

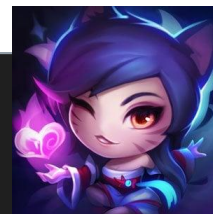
R 鍵：星界躍動 (SpeelyTroubleBubble)

- 暫時傳送至附近新的位置，1 秒後返回原本的位置

```
1  #include <string>
2
3  class Zoe {
4  private:
5      std::string name;
6      int abilityPower;
7      int health;
8  public:
9      /* Getter */
10     std::string GetName();
11     int GetAbilityPower();
12     int GetHealth();
13
14     /* Setter */
15     void SetName();
16     void SetAbilityPower();
17     void SetHealth();
18
19     /* Wizard Only*/
20     void WizardLevelUp();
21
22     /* Zoe-Only Skill */
23     void MoreSparkles();
24     void PaddleStar();
25     void SpellThief();
26     void SpeelyTroubleBubble();
27 };
```

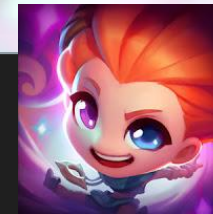

回顧：繼承大概念

- 到這邊會開始覺得不對勁
- 出現了一些重複的 Code



阿璃 法師

```
1  #include <string>
2
3  class Ahri {
4  private:
5      std::string name;
6      int abilityPower;
7      int health;
8  public:
9      /* Getter */
10     std::string GetName();
11     int GetAbilityPower();
12     int GetHealth();
13
14     /* Setter */
15     void SetName();
16     void SetAbilityPower();
17     void SetHealth();
18
19     /* Wizard Only*/
20     void WizardLevelUp();
21
22     /* Ahri-Only Skill */
23     void OrbOfDescription();
24     void FoxFire();
25     void Charm();
26     void SpiritRush();
27 };
```



柔伊 法師

```
1  #include <string>
2
3  class Zoe {
4  private:
5      std::string name;
6      int abilityPower;
7      int health;
8  public:
9      /* Getter */
10     std::string GetName();
11     int GetAbilityPower();
12     int GetHealth();
13
14     /* Setter */
15     void SetName();
16     void SetAbilityPower();
17     void SetHealth();
18
19     /* Wizard Only*/
20     void WizardLevelUp();
21
22     /* Zoe-Only Skill */
23     void MoreSparkles();
24     void PaddleStar();
25     void SpellThief();
26     void SpeelyTroubleBubble();
27 };
```

...

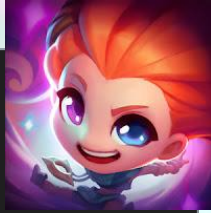
回顧：繼承大概念

- 假設2043年這遊戲還在...法師數量已經高達100隻
- 出現了**大量**重複的 Code
 - 重複撰寫許多次 WizardLevelUp() 的實作是可預期的情況
 - 若我們修改 WizardLevelUp() 的實作, 要改 100 個類別的實作
- 有人想過怎麼解決這個問題嗎?



阿璃 法師

```
1  #include <string>
2
3  class Ahri {
4  private:
5      std::string name;
6      int abilityPower;
7      int health;
8  public:
9      /* Getter */
10     std::string GetName();
11     int GetAbilityPower();
12     int GetHealth();
13
14     /* Setter */
15     void SetName();
16     void SetAbilityPower();
17     void SetHealth();
18
19     /* Wizard Only*/
20     void WizardLevelUp();
21
22     /* Ahri-Only Skill */
23     void OrbOfDescription();
24     void FoxFire();
25     void Charm();
26     void SpiritRush();
27 };
```



柔伊 法師

```
1  #include <string>
2
3  class Zoe {
4  private:
5      std::string name;
6      int abilityPower;
7      int health;
8  public:
9      /* Getter */
10     std::string GetName();
11     int GetAbilityPower();
12     int GetHealth();
13
14     /* Setter */
15     void SetName();
16     void SetAbilityPower();
17     void SetHealth();
18
19     /* Wizard Only*/
20     void WizardLevelUp();
21
22     /* Zoe-Only Skill */
23     void MoreSparkles();
24     void PaddleStar();
25     void SpellThief();
26     void SpeelyTroubleBubble();
27 };
```


嘗試解決問題的方向

- 想一下，有沒有什麼辦法可以解決這個問題？



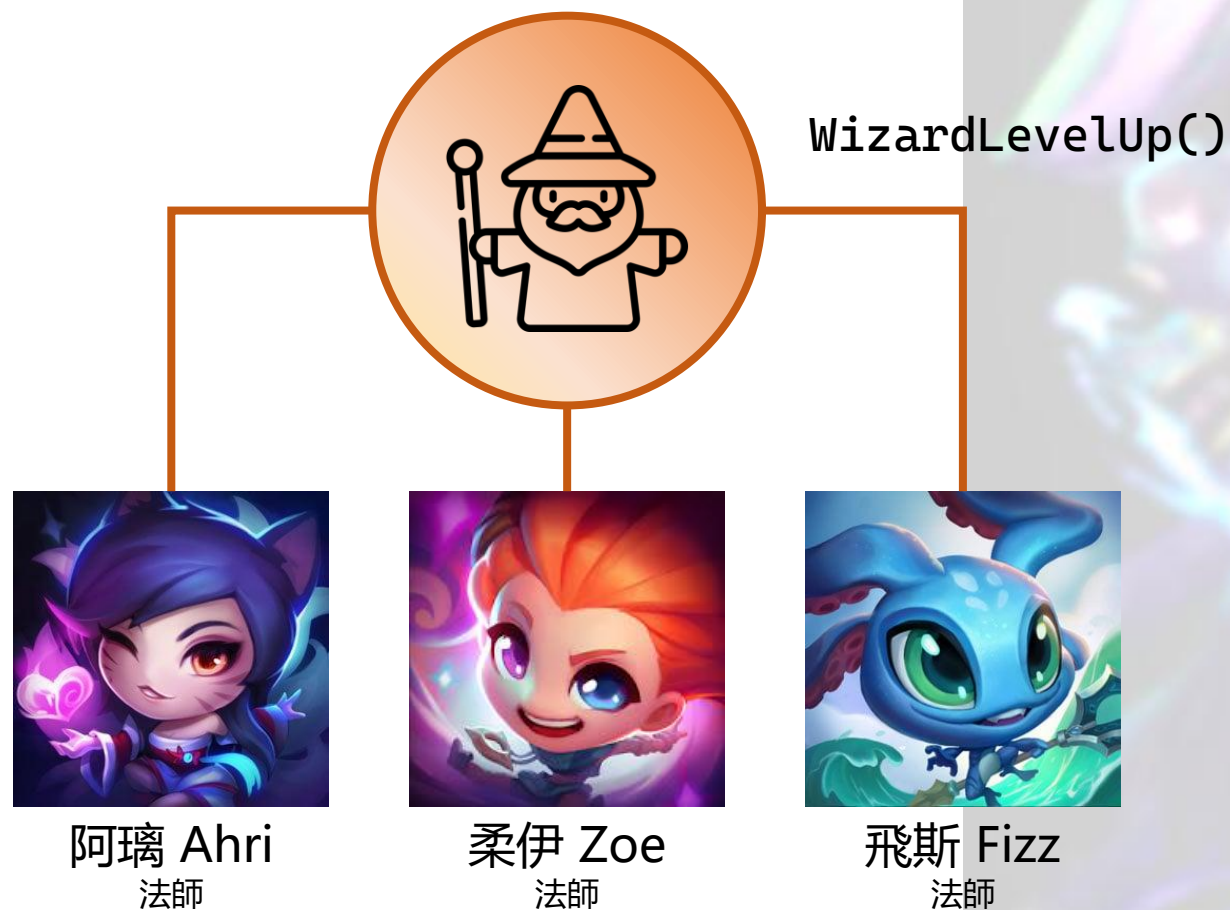
嘗試解決問題的方向

- 我們可以歸納出，這些角色其實都是法師



嘗試解決問題的方向

- 如果我們有辦法可以實踐這個階層圖，也許就會便利許多！



抽象化 (Abstraction)

- 抽象化是 OOP 的核心概念之一，指的是從多個具體的事物中，提取出共同的特徵與行為，並建立一個「抽象層級」的模型
 - 讓我們不用關心細節，也能理解或操作這些事物
 - 將現實世界的物件或概念來「建模」
- 理想上，要先抽象化，再實作



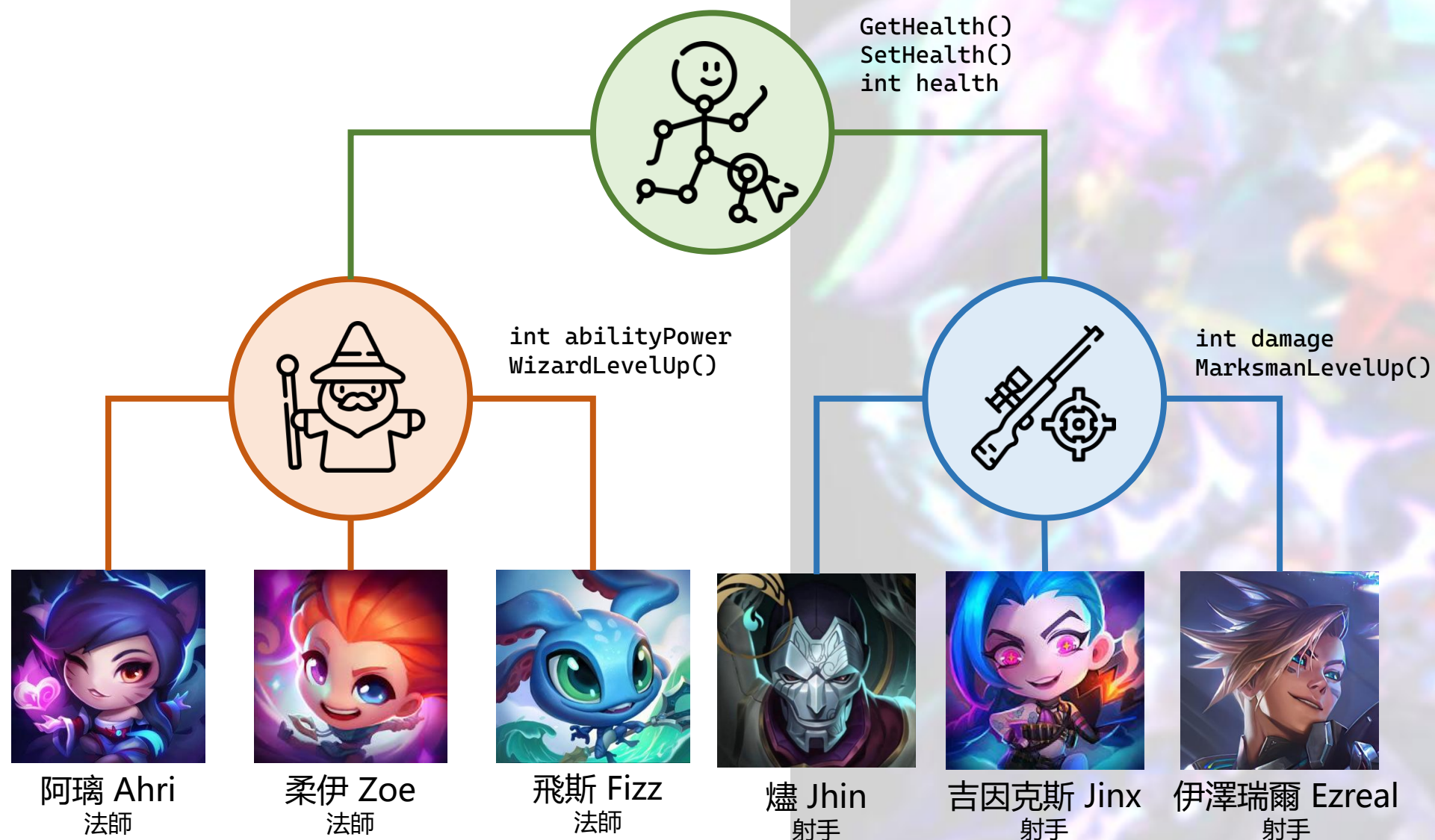
嘗試解決問題的方向

- 用這樣的方式，我們可以統一每個法師 WizardLevelUp() 的實作
- 這樣即使有 100 個法師種類的角色，我們只需要實作角色內專屬的函式即可
- 這樣的方式可以大幅減少重複的程式碼



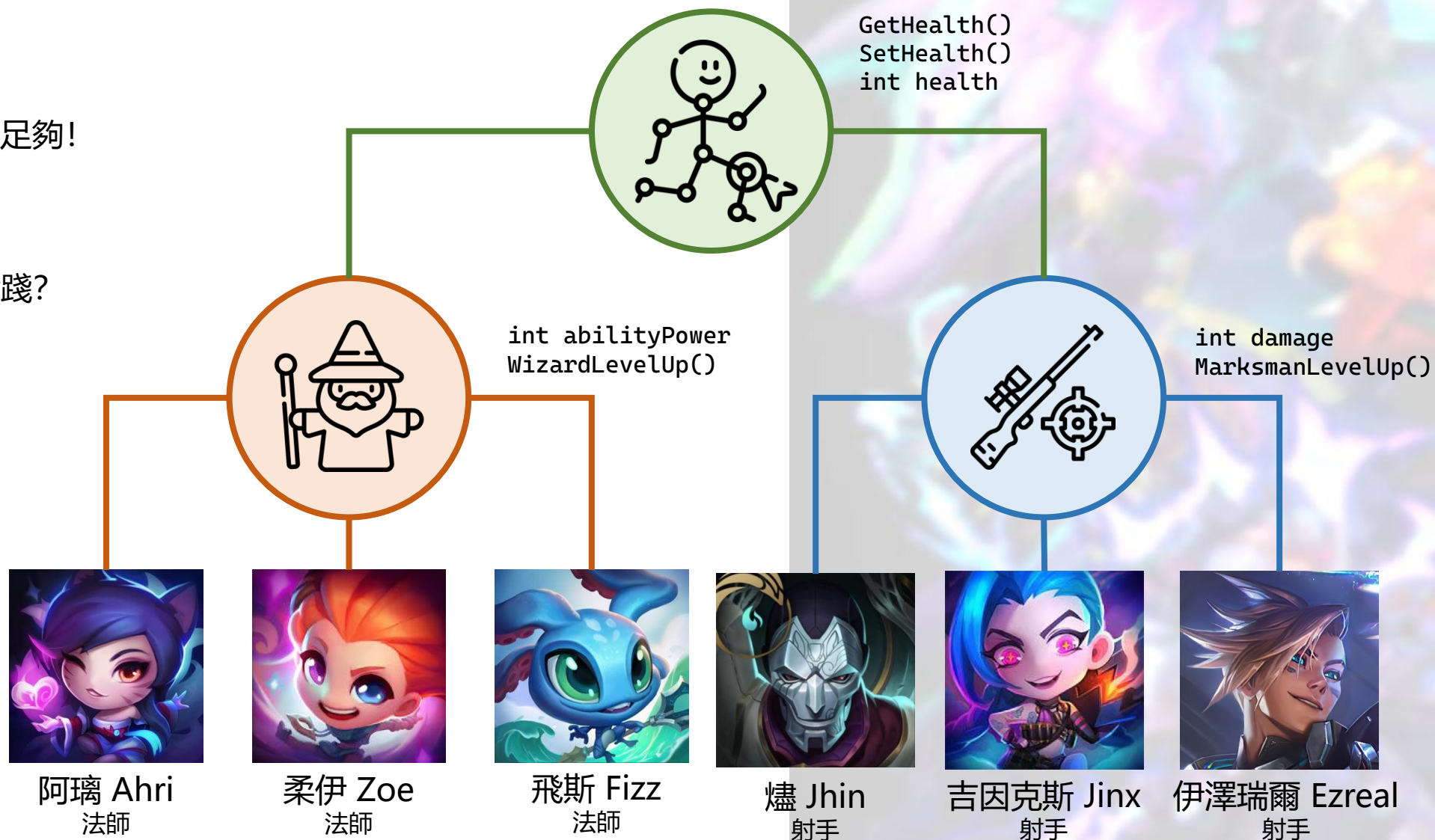
嘗試解決問題的方向

- 我們甚至還可以...



嘗試解決問題的方向

- 看起來更加簡潔且表達力足夠!
- 想法 Approved, 如何實踐?



概念：父類別 (Parent) 與子類別 (Children)

- 在這個章節，我們將延續「問題解決的方向」，繼續推演父類別與子類別的關係。



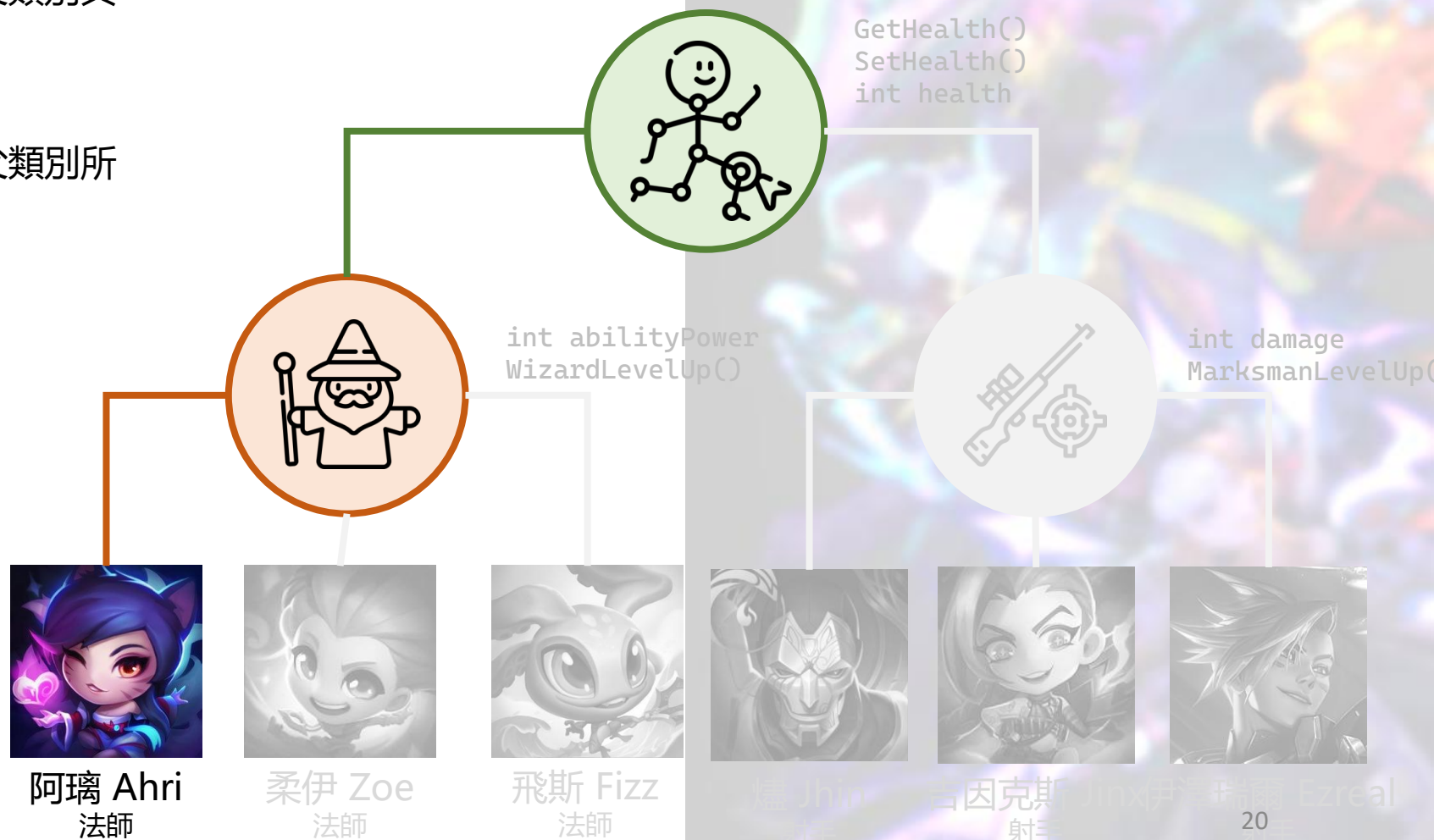
概念：父類別（Parent）與子類別（Children）

- 透過這樣的方式，我們稱為繼承
- 阿璃繼承了法師的特性、法師繼承的角色的特性，**等同於阿璃繼承了法師與角色的特性**
- 阿璃是法師而且是遊戲中一個角色



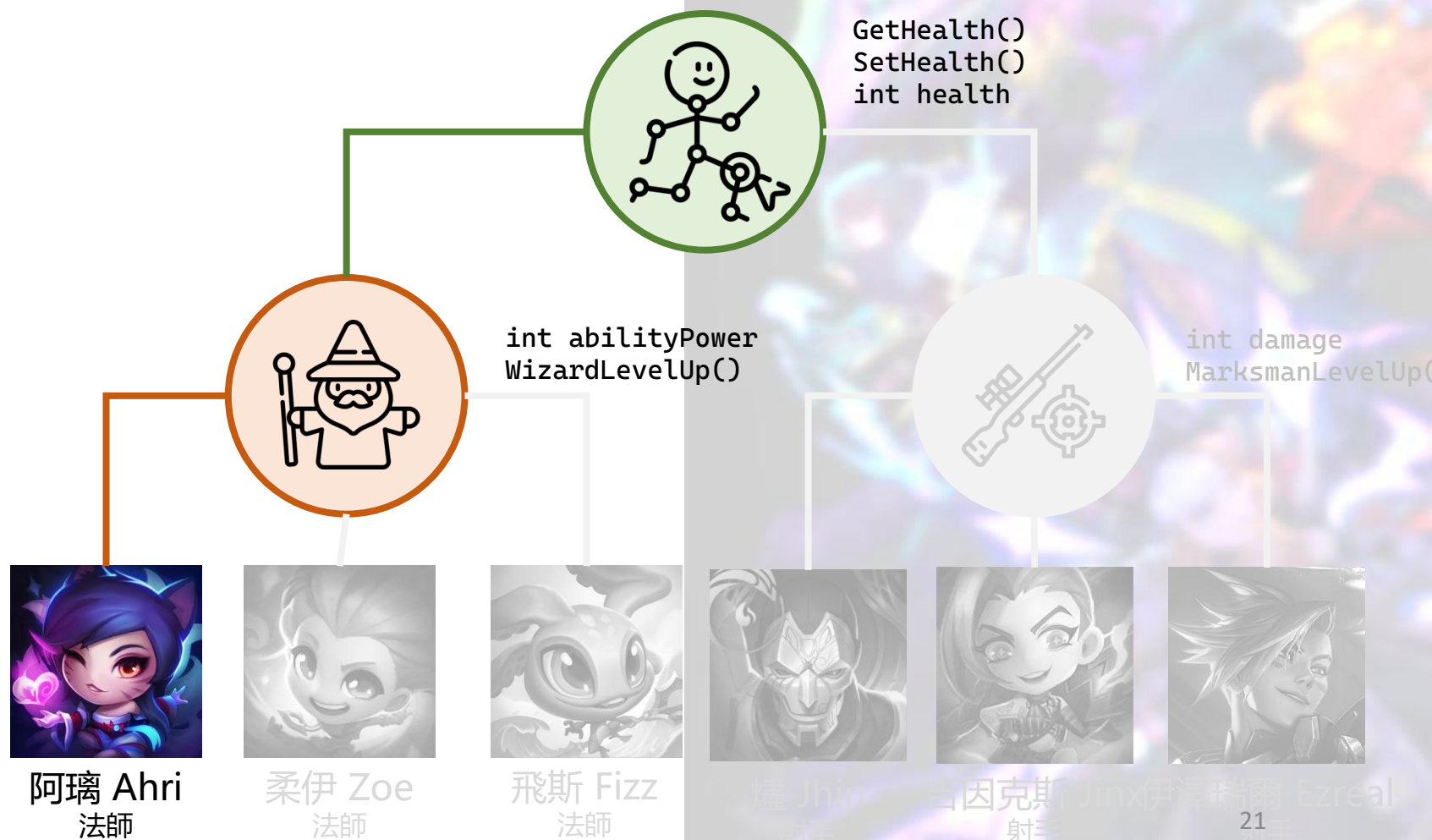
概念：父類別（Parent）與子類別（Children）

- 在這邊，我們可以稍微提一下所謂的父類別與子類別
- 父類別主要為繼承上游，子類別繼承父類別所有的成員與函式



概念：父類別（Parent）與子類別（Children）

- 在這個例子中：
 - 法師是角色的子類別
 - 阿璃是法師的子類別
- 法師具有角色**所有的**成員與函式
- 阿璃具有法師**所有的**成員與函式



概念：父類別（Parent）與子類別（Children）

- 回來看這個程式...
- 實踐一下之前的想法？

```
1  #include <string>
2
3  class Ahri {
4  private:
5      std::string name;
6      int abilityPower;
7      int health;
8  public:
9      /* Getter */
10     std::string GetName();
11     int GetAbilityPower();
12     int GetHealth();
13
14     /* Setter */
15     void SetName();
16     void SetAbilityPower();
17     void SetHealth();
18
19     /* Wizard Only*/
20     void WizardLevelUp();
21
22     /* Ahri-Only Skill */
23     void OrbOfDescription();
24     void FoxFire();
25     void Charm();
26     void SpiritRush();
27 };
```

概念：父類別（Parent）與子類別（Children）

- 以阿璃來實踐一下想法。

```
1 #include <string>
2
3 class Ahri {
4 private:
5     std::string name;
6     int abilityPower;
7     int health;
8 public:
9     /* Getter */
10    std::string GetName();
11    int GetAbilityPower();
12    int GetHealth();
13
14    /* Setter */
15    void SetName();
16    void SetAbilityPower();
17    void SetHealth();
18
19    /* Wizard Only*/
20    void WizardLevelUp();
21
22    /* Ahri-Only Skill */
23    void OrbOfDescription();
24    void FoxFire();
25    void Charm();
26    void SpiritRush();
27 };
```

=

```
1 #include <string>
2
3 #include "mage.h"
4
5 class Ahri : Mage {
6 public:
7     /* Ahri-Only Skill */
8     void OrbOfDescription();
9     void FoxFire();
10    void Charm();
11    void SpiritRush();
12 };
```

阿璃特有的技能。

```
1 #include "character.h"
2
3 class Mage : Character {
4 private:
5     int abilityPower;
6 public:
7     /* Getter */
8     int GetAbilityPower();
9
10    /* Setter */
11    void SetAbilityPower();
12
13    /* Wizard-only */
14    void WizardLevelUp();
15 };
```

我們假設法師都會有魔法攻擊，升級時針對魔法攻擊作調整。

```
1 #include <string>
2
3 class Character {
4 private:
5     std::string name;
6     int health;
7 public:
8     /* Getter */
9     std::string GetName();
10    int GetHealth();
11
12    /* Setter */
13    void SetName();
14    void SetHealth();
15 };
```

我們假設角色都會有名字與血量。

概念：父類別（Parent）與子類別（Children）

- 加上柔依呢？

```
1  #include <string>
2
3  class Zoe {
4  private:
5      std::string name;
6      int abilityPower;
7      int health;
8  public:
9      /* Getter */
10     std::string GetName();
11     int GetAbilityPower();
12     int GetHealth();
13
14     /* Setter */
15     void SetName();
16     void SetAbilityPower();
17     void SetHealth();
18
19     /* Wizard Only*/
20     void WizardLevelUp();
21
22     /* Zoe-Only Skill */
23     void MoreSparkles();
24     void PaddleStar();
25     void SpellThief();
26     void SpeedyTroubleBubble();
27 };
```

=

```
1  #include <string>
2
3  #include "mage.h"
4
5  class Ahri : Mage {
6  public:
7      /* Ahri-Only Skill */
8      void OrbOfDescription();
9      void FoxFire();
10     void Charm();
11     void SpiritRush();
12 };
```

```
1  #include <string>
2
3  #include "mage.h"
4
5  class Zoe : Mage {
6  public:
7      /* Zoe-Only Skill */
8      void MoreSparkles();
9      void PaddleStar();
10     void SpellThief();
11     void SpeedyTroubleBubble();
12 };
```

```
1  #include "character.h"
2
3  class Mage : Character {
4  private:
5      int abilityPower;
6  public:
7      /* Getter */
8      int GetAbilityPower();
9
10     /* Setter */
11     void SetAbilityPower();
12
13     /* Wizard-only */
14     void WizardLevelUp();
15 };
```

```
1  #include <string>
2
3  class Character {
4  private:
5      std::string name;
6      int health;
7  public:
8      /* Getter */
9      std::string GetName();
10     int GetHealth();
11
12     /* Setter */
13     void SetName();
14     void SetHealth();
15 };
```

程式碼重複率近乎 0！

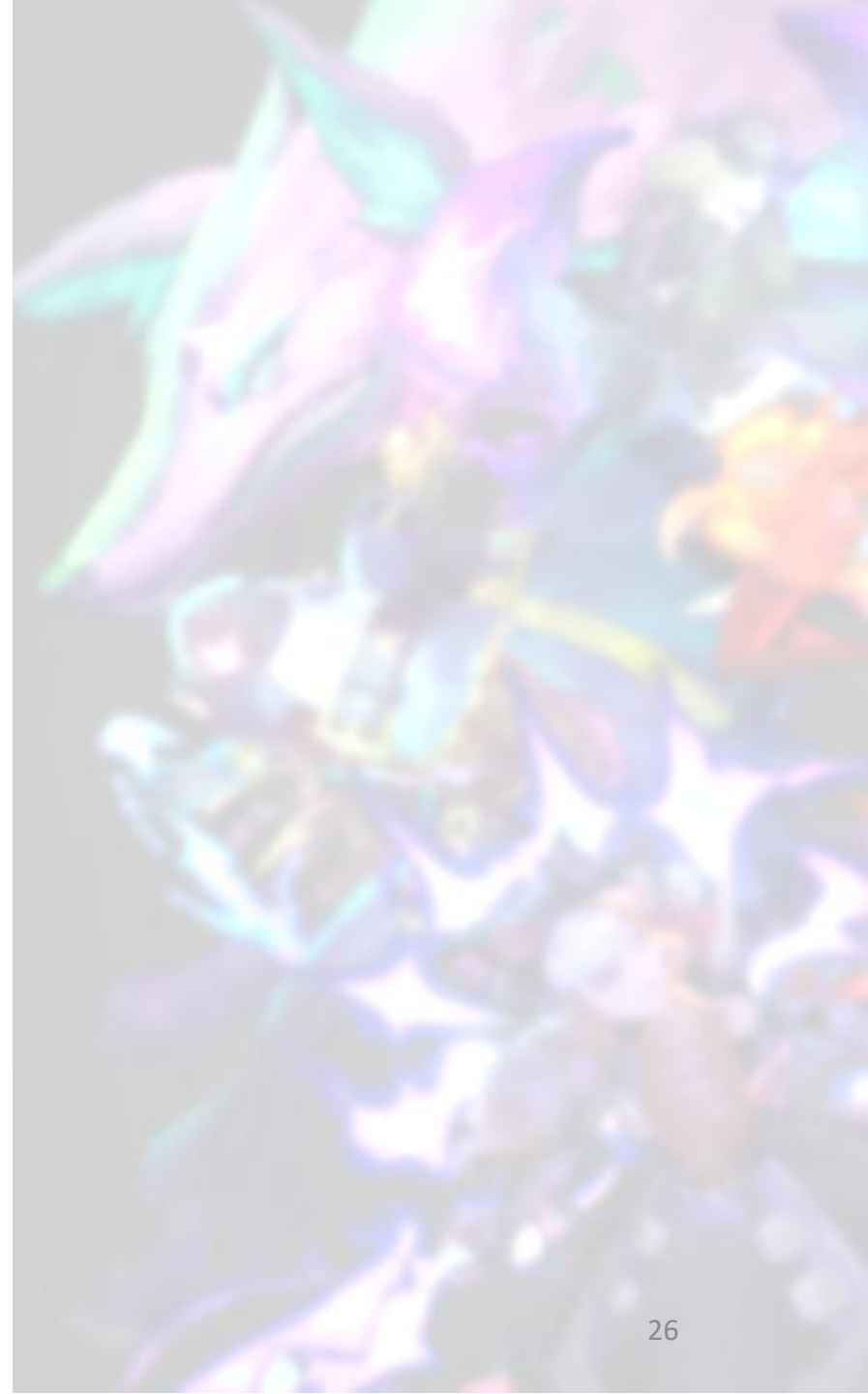
概念：父類別（Parent）與子類別（Children）

- 看完前面的精采操作之後



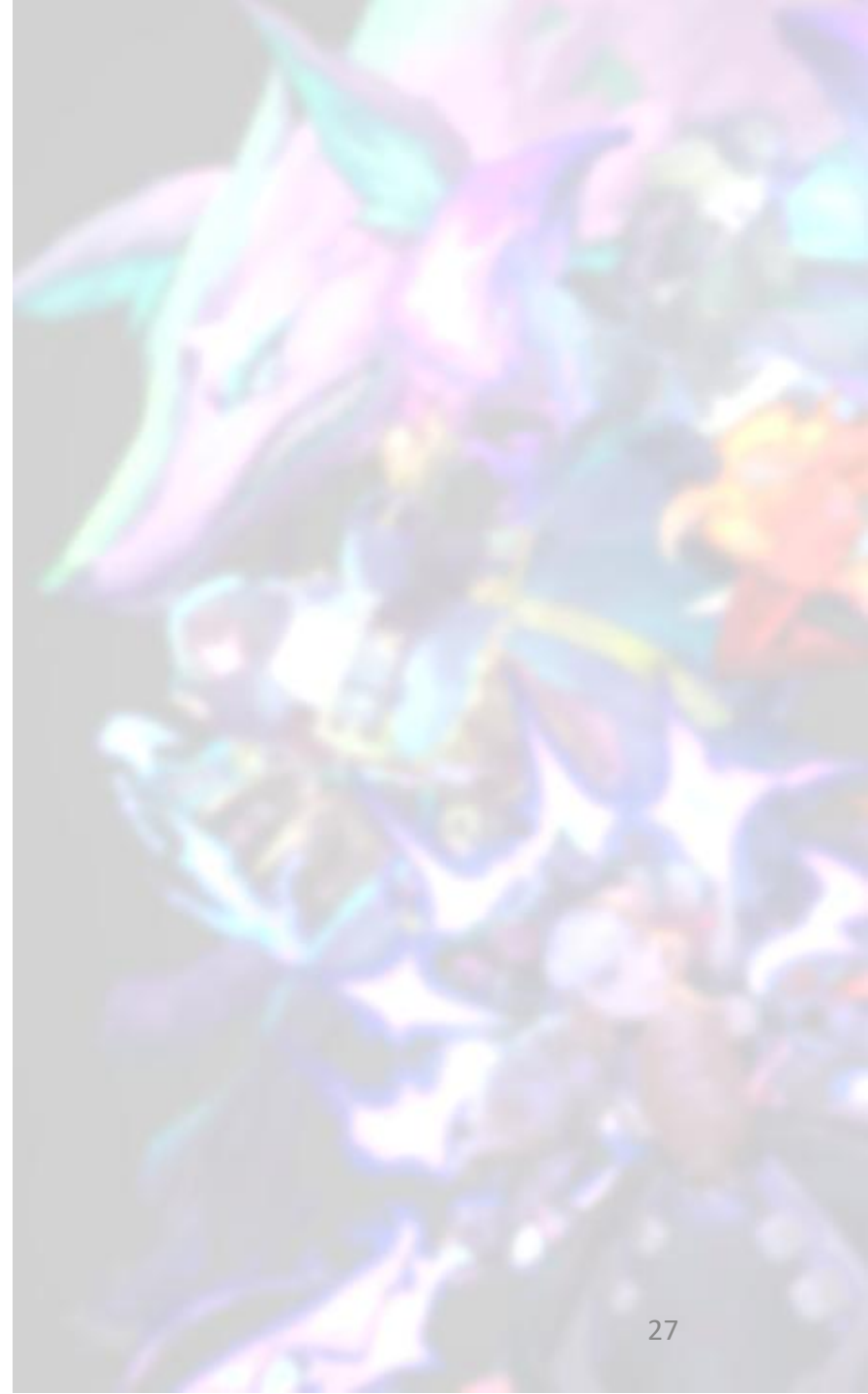
繼承關係

- 不同類別之間的關係
 - 合成關係 vs. **繼承關係**
- 合成關係 (has-a)
 - 物件內含一或多個其他類別的物件
 - Ex. 車子有多個輪子 (輪子屬於車子的物件)
- **繼承關係 (is-a)**
 - **衍生類別 “is-a” 基礎類別**
 - **Ex. 貓咪是一種動物**
 - **把貓咪看成一種動物**
 - **貓咪具有動物的屬性與行為**



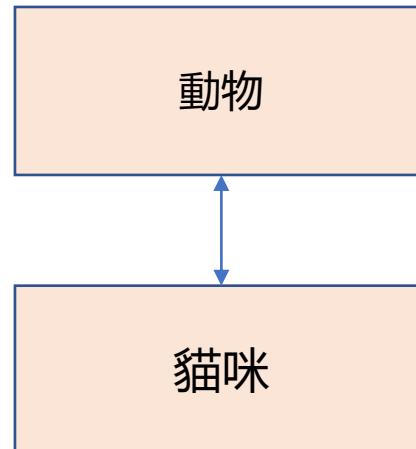
繼承的好處

- 軟體（或程式碼）重複使用
- 使用舊的類別建構新類別
 - 站在現有的基礎上
 - 可擁有現有類別的成員資料與行為
 - 可改寫現有類別的成員資料與行為
 - 可外加新的成員資料與行為



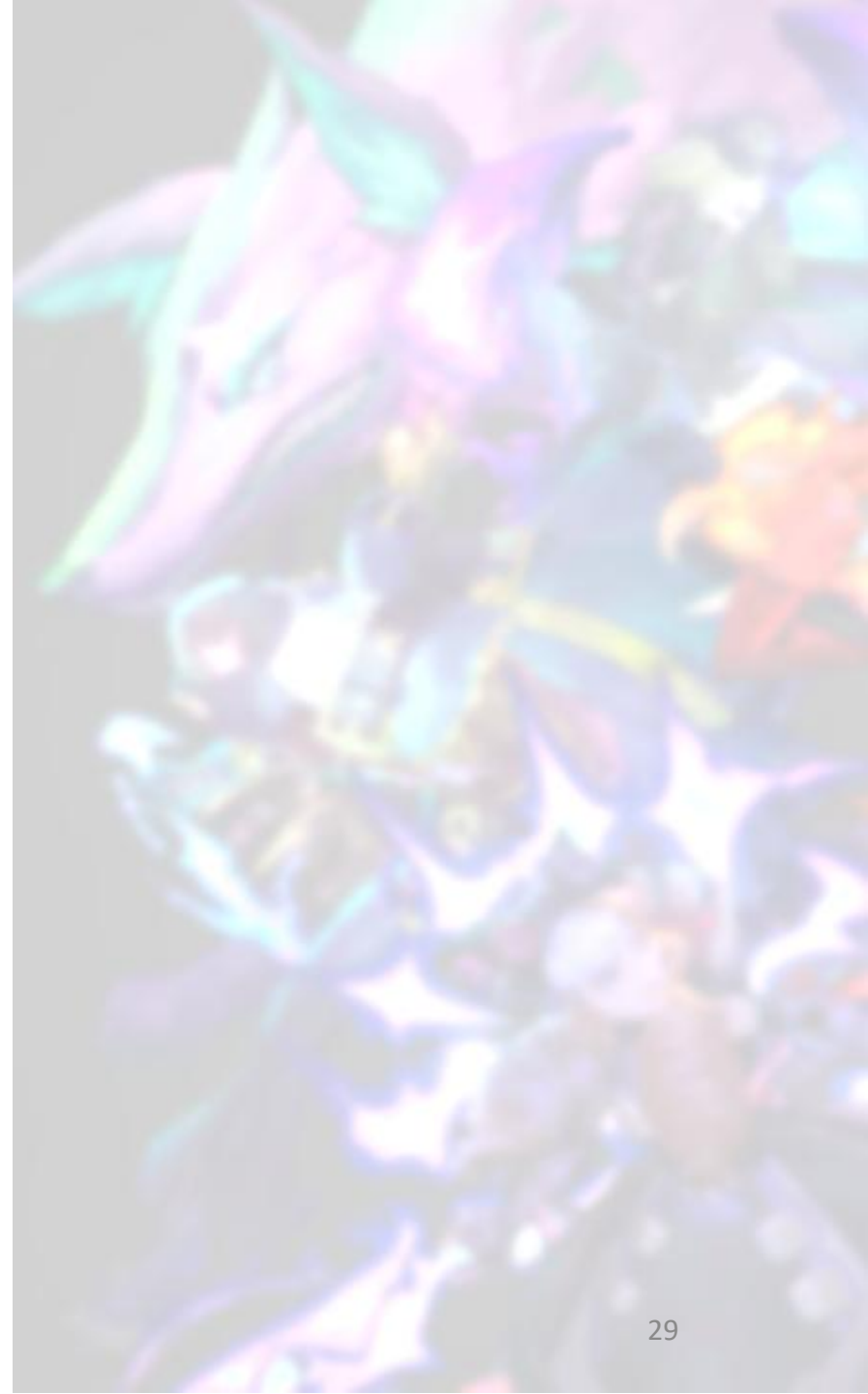
基礎類別與衍生類別

- 被繼承的類別稱為基礎類別 (Base Class)
- 繼承別人的類別稱為衍生類別 (Derived Class)
- 衍生類別的物件也是基礎類別的物件
 - Ex. 貓咪類別繼承自動物
 - 貓咪的物件也是動物的物件
 - 動物是基礎類別
 - 貓咪是衍生類別



範圍比較

- 基礎類別包含的範圍比衍生類別廣
- Ex.
 - 基礎類別：載具 (Vehicle)
 - 汽車、卡車、船、腳踏車、...
 - 衍生類別：汽車 (Car)
 - 雙門轎車、四門轎車、超跑、電動車、...



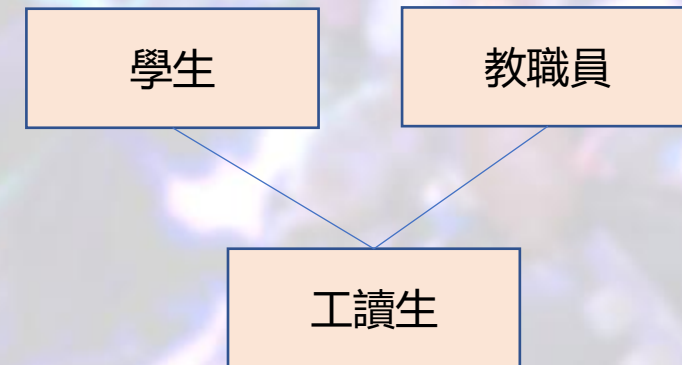
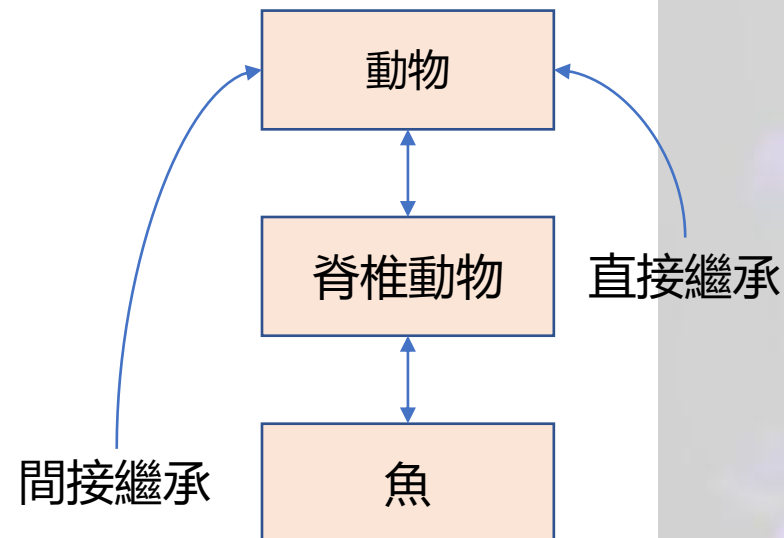
繼承的例子

基礎類別	衍生類別
學生	國小生 國中生 高中生 大學生 研究生 博士生
形狀	圓形 三角形 四邊形
貸款	學生貸款 汽車貸款 房屋貸款
員工	教職員 行政人員 警衛 清潔人員

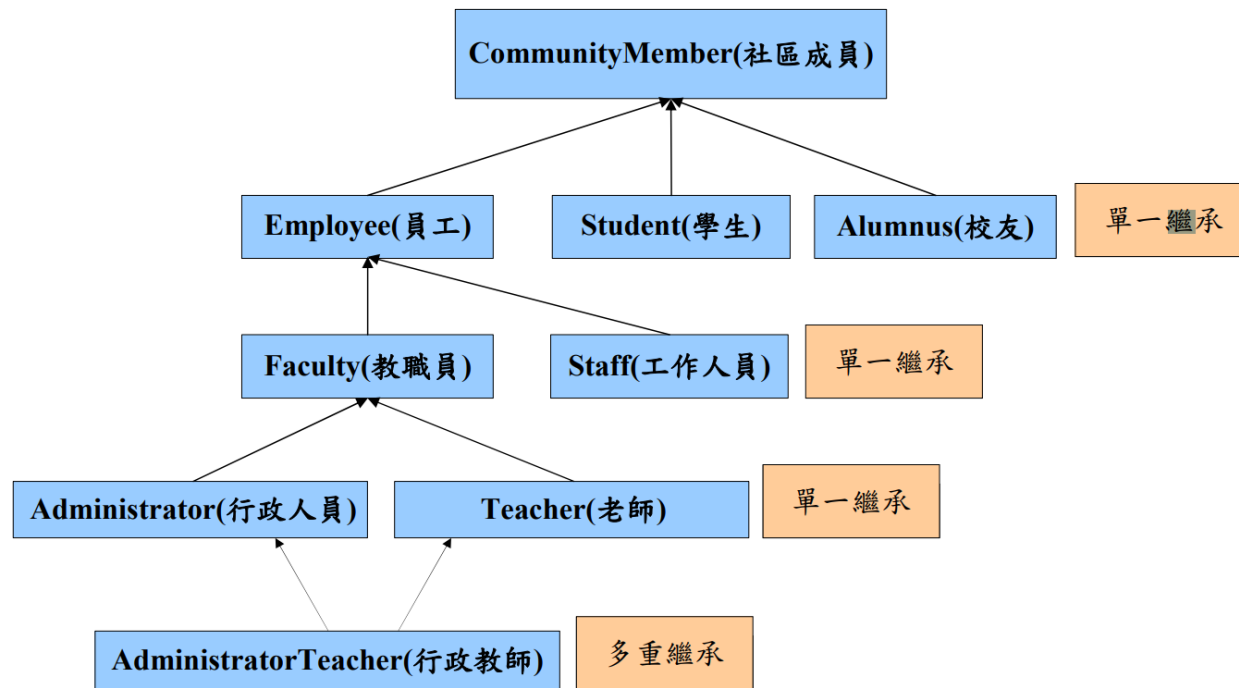
繼承階層圖 (Inheritance hierarchy)

- 繼承關係：樹狀階層圖
- 每個類別可以是
 - 基礎類別
 - 提供其他類別繼承
 - 提供成員與行為給其衍生類別
 - 衍生類別
 - 繼承其他類別
 - 可以使用基礎類別的成員與行為

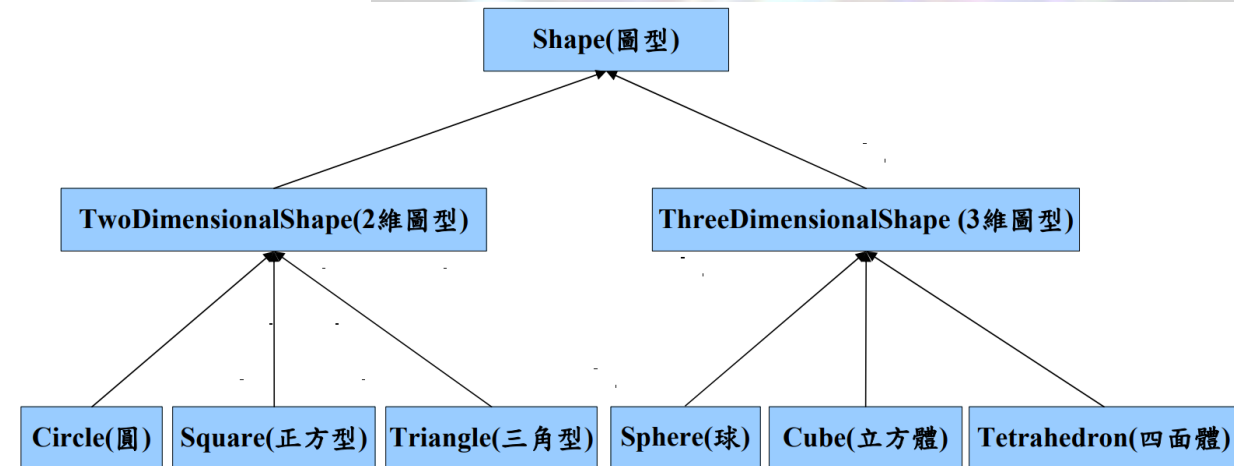
- 繼承關係分為：
 - 直接 (Direct) 繼承
 - 間接 (Indirect) 繼承
 - 單一 (Single) 繼承
 - 繼承自單一個基礎類別
 - 多重 (Multiple) 繼承
 - 繼承自多個基礎類別



繼承階層圖 (Inheritance hierarchy)



大學社群繼承階層圖



形狀繼承階層圖

概念：父類別（Parent）與子類別（Children）

- 看起來很酷，不過... 我要怎麼初始化這些成員？

```
1  #include <string>
2
3  #include "mage.h"
4
5  class Zoe : Mage {
6  public:
7      /* Zoe-Only Skill */
8      void MoreSparkles();
9      void PaddleStar();
10     void SpellThief();
11     void SpeelyTroubleBubble();
12 };
```

```
1  #include "character.h"
2
3  class Mage : Character {
4  private:
5      int abilityPower;
6  public:
7      /* Getter */
8      int GetAbilityPower();
9
10     /* Setter */
11     void SetAbilityPower();
12
13     /* Wizard-only */
14     void WizardLevelUp();
15 };
```

```
1  #include <string>
2
3  class Character {
4  private:
5      std::string name;
6      int health;
7  public:
8      /* Getter */
9      std::string GetName();
10     int GetHealth();
11
12     /* Setter */
13     void SetName();
14     void SetHealth();
15 };
```

初始化列表 (Initialize List) 與為何需要初始化列表

- 在這個章節，我們將嘗試解決繼承在初始化成員的行為。



初始化列表 (Initialize List) 與為何需要初始化列表

- 如果我們幫阿璃加上建構子，你會發現...

```
1  #include <string>
2
3  #include "mage.h"
4
5  class Ahri : Mage {
6  public:
7      Ahri(int abilityPower, std::string name, int health);
8      /* Ahri-Only Skill */
9      void OrbOfDescription();
10     void FoxFire();
11     void Charm();
12     void SpiritRush();
13 };
```

- abilityPower 是 Mage 的私有成員，所以 Ahri 無法存取

```
Ahri::Ahri(int abilityPower, std::string name, int health) {
    this->abilityPower = abilityPower
}
void Ahri: mage.h(5, 9): Declared private here
```

'abilityPower' is a private member of 'Mage' clang(access)

Class Character	Class Mage
<pre>1 #include <string> 2 3 class Character { 4 private: 5 std::string name; 6 int health; 7 public: 8 Character(std::string name, int health); 9 10 /* Getter */ 11 std::string GetName(); 12 int GetHealth(); 13 14 /* Setter */ 15 void SetName(std::string name); 16 void SetHealth(int health); 17 };</pre>	<pre>1 #include "character.h" 2 3 class Mage : Character { 4 private: 5 int abilityPower; 6 public: 7 Mage(int abilityPower, std::string name, int health); 8 9 /* Getter */ 10 int GetAbilityPower(); 11 12 /* Setter */ 13 void SetAbilityPower(int abilityPower); 14 15 /* Wizard-only */ 16 void WizardLevelUp(); 17 };</pre>

初始化列表 (Initialize List) 與為何需要初始化列表

- 想一下解決方法?
- Character 與 Mage 通常都會有建構子
 - Character 一定會需要血量與名稱，所以建構子一定要有血量與名稱
 - Mage 一定會需要魔法攻擊、血量與名稱，所以建構子一定要有魔法攻擊、血量與名稱

```
Class Character
1  #include <string>
2
3  class Character {
4  private:
5      std::string name;
6      int health;
7  public:
8      Character(std::string name, int health);
9
10     /* Getter */
11     std::string GetName();
12     int GetHealth();
13
14     /* Setter */
15     void SetName(std::string name);
16     void SetHealth(int health);
17 };
```

```
Class Mage
1  #include "character.h"
2
3  class Mage : Character {
4  private:
5      int abilityPower;
6  public:
7      Mage(int abilityPower, std::string name, int health);
8
9      /* Getter */
10     int GetAbilityPower();
11
12     /* Setter */
13     void SetAbilityPower(int abilityPower);
14
15     /* Wizard-only */
16     void WizardLevelUp();
17 };
```

初始化列表 (Initialize List) 與為何需要初始化列表

- 如果我們可以透過建構子來進行初始化呢?
 - Mage 自己私有的成員在自己的建構子程式區塊中初始化
 - Mage 使用 **Character** 的建構子來初始化 Character 的血量與名稱

```
Class Character
1  #include <string>
2
3  class Character {
4  private:
5      std::string name;
6      int health;
7  public:
8      Character(std::string name, int health):
9
10         /* Getter */
11         std::string GetName();
12         int GetHealth();
13
14         /* Setter */
15         void SetName(std::string name);
16         void SetHealth(int health);
17     };

```

```
Class Mage
1  #include "character.h"
2
3  class Mage : Character {
4  private:
5      int abilityPower;
6  public:
7      Mage(int abilityPower, std::string name, int health);
8
9      /* Getter */
10     int GetAbilityPower();
11
12     /* Setter */
13     void SetAbilityPower(int abilityPower);
14
15     /* Wizard-only */
16     void WizardLevelUp();
17 };

```

初始化列表 (Initialize List) 與為何需要初始化列表

原本要初始化的參數

```
1  Mage::Mage(int abilityPower, std::string name, int health) : Character(name, health){
2      this->abilityPower = abilityPower;
3  }
```

- 上圖的 “: Character(name, health)”, 我們稱為 Initialize List
 - 冒號後面代表指定哪個父類別的建構子
- 使用 Character 的建構子來初始化 Character 的成員, 讓 Character 的成員保持私有又能夠被子類給初始化

初始化列表 (Initialize List) 與為何需要初始化列表

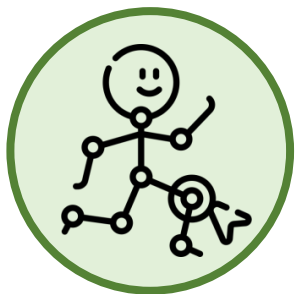
- 但不能是這樣！

```
1 Mage::Mage(int abilityPower, std::string name, int health){  
2     Character(name, health);  
3     this->abilityPower = abilityPower;  
4 }
```

- 這個動作等同於
 - 建構一個 Character 物件，與我們預期要初始化父類成員是完全不同的結果

初始化列表 (Initialize List) 與為何需要初始化列表

- 回到阿璃的例子，我們要怎麼在建構阿璃時初始化阿璃的父類 Mage 與 Character 呢？



```
1 Character::Character(std::string name, int health) {  
2     this->name = name;  
3     this->health = health;  
4 }
```



```
1 Mage::Mage(int abilityPower, std::string name, int health) : Character(name, health){  
2     this->abilityPower = abilityPower;  
3 }
```

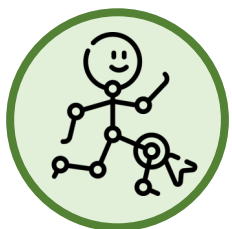


阿璃 Ahri

```
1 Ahri::Ahri(int abilityPower, std::string name, int health) : Mage(abilityPower, name, health) {  
2  
3 }
```

順序問題

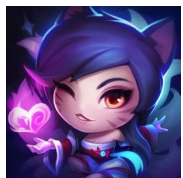
- 初始化衍生類別物件
 - 導致一連串的建構子呼叫與處理
 - 衍生類別建構子會呼叫基礎類別建構子



```
1 Character::Character(std::string name, int health) {  
2     this->name = name;  
3     this->health = health;  
4 }
```



```
1 Mage::Mage(int abilityPower, std::string name, int health) : Character(name, health){  
2     this->abilityPower = abilityPower;  
3 }
```



阿璃 Ahri

```
1 Ahri::Ahri(int abilityPower, std::string name, int health) : Mage(abilityPower, name, health) {  
2  
3 }
```

呼叫順序

完成順序

③

①

②

②

①

③

權限控制：protected

- 在這個章節，我們將嘗試解決無法存取父類成員的問題。



權限控制: protected

- 還記得 WizardLevelUp() 這個函數嗎，我們要開始設計這個函數
- 如果要設計 WizardLevelUp() 這個函數，原先的code少了什麼？
 - 但由於角色還沒有 Level 這個成員，所以我們把它加上去

```
1  #include <string>
2
3  class Character {
4  private:
5      std::string name;
6      int health;
7
8  public:
9      Character(std::string name, int health);
10
11     /* Getter */
12     std::string GetName();
13     int GetHealth();
14
15
16     /* Setter */
17     void SetName(std::string name);
18     void SetHealth(int health);
19
20 };
```


權限控制：protected

- 接下來我們開始設計 WizardLevelUp() 這個函數
- 請問WizardLevelUp()應該實作在哪一份code中?
 - 在Mage.hpp/Mage.cpp
 - 我們期望每升級一等，角色會得到 $Level \times 0.5 \times AbilityPower$ 的魔力加成

```
/* Getter */
int Mage::
    return
}

/* Setter
void Mage:
    this->
}

/* Wizard-
void Mage:
    this->level * 0.5 * abilityPower;
}
```

'level' is a private member of 'Character' clang(access)
character.h(7, 9): Declared private here

field level

Type: int
初始化，角色 Level 一開始為 1。

// In Character
private: int level = 1

View Problem (Alt+F8) No quick fixes available

- 我們沒有辦法直接拿到 level，因為 level 是 Character 的 private member
- 怎麼辦呢？

權限控制：protected

- 我們把 Character 中的 level 設定成 public，完成！

```
/* Getter */
int Mage::
    return
}

/* Setter
void Mage:
    this->
}

/* Wizard-
void Mage:
    this->level * 0.5 * abilityPower;
}
```

'level' is a private member of 'Character' clang(access)
character.h(7, 9): Declared private here

field level

Type: int
初始化，角色 Level 一開始為 1。

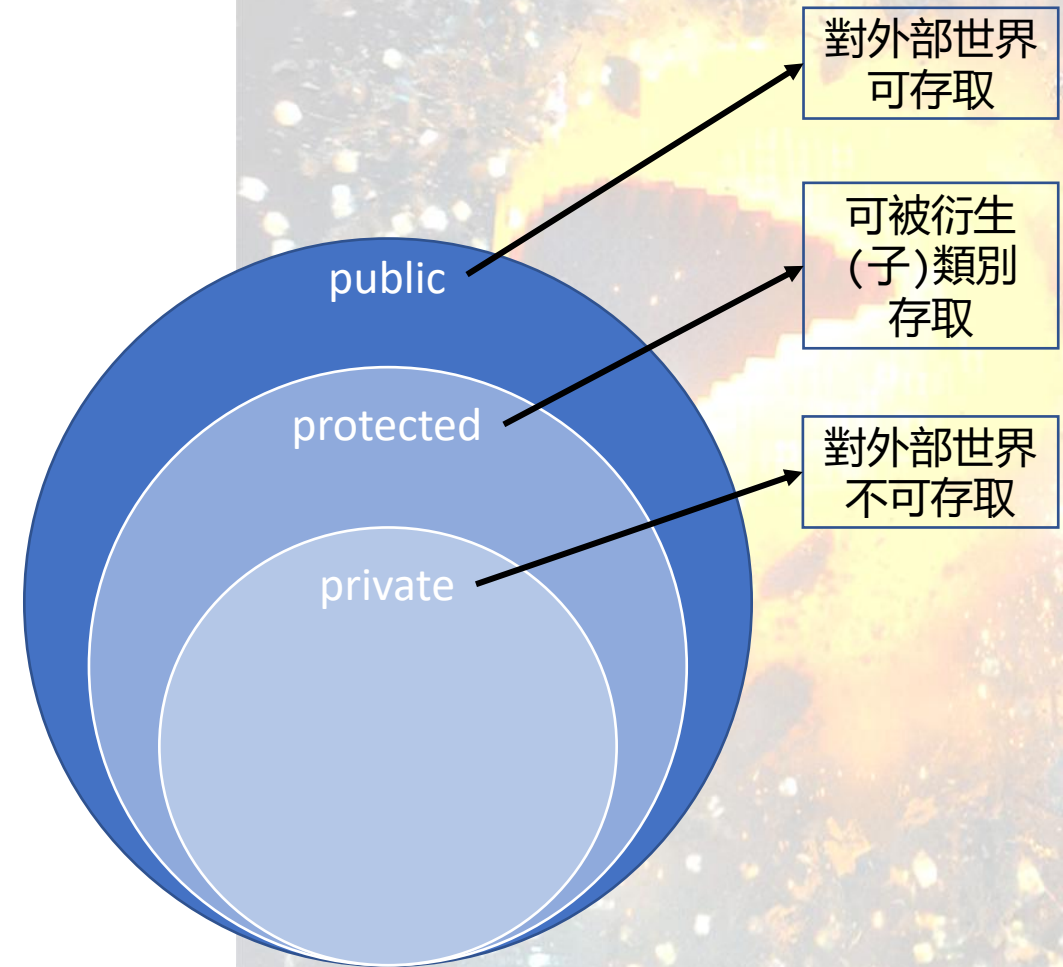
// In Character
private: int level = 1

[View Problem \(Alt+F8\)](#) No quick fixes available

- 但我們不可能讓 Character 的 level 直接 public，這樣違反了封裝性
- 有沒有什麼辦法可以「只讓子類別存取、不讓外部存取」呢？

封裝：存取修飾字

- 要做到資訊隱藏 (Information hiding) 可以使用存取修飾字
- 分為：
 - Public: 是指對存取權限完全的公開，任何物件都可以存取
 - Private: 是限制最嚴格的access modifier，只有在類別本身內部可以存取
 - **Protected: 存取是受限的，除了類別自己可以使用外，只有其成員函式與類別的朋友 (friend) 以及子類別可以存取**
 - 「繼承」的時候介紹



權限控制: protected

- 對於這個例子，我們可以使用 protected 來使父類的成員能夠被子類所存取
- 透過這樣的方式，Mage 就能夠取得 Character 的 Level，進行 WizardLevelUp() 的運算

```
1  /* Wizard-only */
2  void Mage::WizardLevelUp(){
3      this->level * 0.5 * abilityPower;
4  }
```

- 但...這樣就結束了嗎?

```
1  #include <string>
2
3  class Character {
4  private:
5      std::string name;
6      int health;
7  protected:
8      int level = 1; // 初始化，角色 Level 一開始為 1。
9  public:
10     Character(std::string name, int health);
11
12     /* Getter */
13     std::string GetName();
14     int GetHealth();
15     int GetLevel(); // 取得等級
16
17     /* Setter */
18     void SetName(std::string name);
19     void SetHealth(int health);
20     void LevelUp(); // 升級
21 };
```


權限控制：protected

- 記得封裝性嗎？如果我們讓 level 放入 protected 其實依然是稍微違反封裝性的
 - 子類別可以直接更改父類別的內部狀態，這可能會導致數據不一致或其他不可預見的錯誤
 - 因為子類能夠任意改動 level 的值，對於「我們希望 Mage 僅讀 level 的值」意願是違反的
- 所以我們應該要繼續遵循使用函數來取得 level 值，也就是從 GetLevel() 下手

```
1  #include <string>
2
3  class Character {
4  private:
5      std::string name;
6      int health;
7      int level = 1; // 初始化，角色 Level 一開始為 1。
8  public:
9      Character(std::string name, int health);
10
11     /* Getter */
12     std::string GetName();
13     int GetHealth();
14     int GetLevel(); // 取得等級
15
16     /* Setter */
17     void SetName(std::string name);
18     void SetHealth(int health);
19     void LevelUp(); // 升級
20 };
```

權限控制：protected

- 記得封裝性嗎？如果我們讓 level 放入 protected 其實依然是稍微違反封裝性的。
 - 子類別可以直接更改父類別的內部狀態，這可能會導致數據不一致或其他不可預見的錯誤
 - 因為子類能夠任意改動 level 的值，對於「我們希望 Mage 僅讀 level 的值」意願是違反的
- 所以我們應該要繼續遵循使用函數來取得 level 值，也就是從 GetLevel() 下手
- 那為什麼我們需要 protected？
 - 若我們希望**某函式**不應該被外部存取，但希望能夠在子類中被存取，我們應該幫該函式設為 protected
 - 這樣不違反封裝性，但又能體現 protected 的用意

```
1  #include <string>
2
3  class Character {
4  private:
5      std::string name;
6      int health;
7      int level = 1; // 初始化，角色 Level 一開始為 1。
8  public:
9      Character(std::string name, int health);
10
11     /* Getter */
12     std::string GetName();
13     int GetHealth();
14     int GetLevel(); // 取得等級
15
16     /* Setter */
17     void SetName(std::string name);
18     void SetHealth(int health);
19     void LevelUp(); // 升級
20 };
```

權限控制: protected (Bonus)



Manager

新遊戲角色目標:
「放越多次技能, 法術傷害越高」
or
「每放100次Q技能, 法術傷害+1」

- 考慮到我們希望在遊戲上, **統計使用技能的數量**, 並在未來供 WizardLevelUp() 當作提升數值使用
- 所以 Character 紀錄了使用技能的數量 (SkillUsageCount)

```
1  //
2  // Created by 黃漢軒 on 2023/10/9.
3  //
4
5  #ifndef OOP_CHARACTER_HPP
6  #define OOP_CHARACTER_HPP
7
8  #include <string>
9
10 class Character {
11 private:
12     std::string name;
13     int health;
14 public:
15     Character(std::string name, int health);
16
17     std::string GetName();
18     int GetHealth();
19
20     void SetName(std::string name);
21     void SetHealth(int health);
22
23     void SkillUsageCount();
24 };
25
26 #endif // OOP_CHARACTER_HPP
27
```

(更正: 回傳不會是void)

權限控制：protected (Bonus)

- 但是 SkillUsageCount 是一個必須要公開的東西嗎？
 - 你玩LOL會去看QWER放過幾次嗎？
- 一般來說，玩家不需要知道技能被用了幾次
 - SkillUsageCount 的用意單純只是想要在 Mage 中用來當作升級時增加魔力值的依據
- 因此，這個 Function 不應該被公開，但應該要能夠被
 - Mage類別存取（用於WizardLevelUp()）
 - 特定子類別所存取（用於特殊角色）

```
1  //
2  // Created by 黃漢軒 on 2023/10/9.
3  //
4
5  #ifndef OOP_CHARACTER_HPP
6  #define OOP_CHARACTER_HPP
7
8  #include <string>
9
10 class Character {
11 private:
12     std::string name;
13     int health;
14 public:
15     Character(std::string name, int health);
16
17     std::string GetName();
18     int GetHealth();
19
20     void SetName(std::string name);
21     void SetHealth(int health);
22
23     void SkillUsageCount();
24 };
25
26 #endif // OOP_CHARACTER_HPP
27
```

權限控制：protected (Bonus)

- 在這個例子下，SkillUsageCount() 就應該被放到 protected
 - 就能夠在不公開這個 function 的情況下
 - 依然被子類別所存取
- 符合「最小公開原則」 (Principle of Least Privilege, POLP)
 - 該原則指出，安全架構的設計應確保每個實體僅獲得執行其功能所需的最少系統資源和授權
 - 重要的安全程式設計原則之一

```
1  //
2  // Created by 黃漢軒 on 2023/10/9.
3  //
4
5  #ifndef OOP_CHARACTER_HPP
6  #define OOP_CHARACTER_HPP
7
8  #include <string>
9
10 class Character {
11 private:
12     std::string name;
13     int health;
14 public:
15     Character(std::string name, int health);
16
17     std::string GetName();
18     int GetHealth();
19
20     void SetName(std::string name);
21     void SetHealth(int health);
22     protected:
23         void SkillUsageCount();
24 };
25
26 #endif // OOP_CHARACTER_HPP
27
```

Protected資料成員的優缺點

- 優點：
 - 衍生類別可直接存取
 - 程式執行效率較高
 - 因為沒有使用SETTER/GETTER
- 缺點：
 - 值沒有經過set/get函式驗證
 - 衍生類別直接存取可能指派不合法的值
 - 實作相關性高
 - 衍生類別的成員函式與基礎類別的實作相關性較高
 - 基礎類別的實作方式改變,衍生類別必須跟著變

繼承與權限控制

繼承與權限控制

- 在這個章節，我們將繼續描述繼承與權限相關的事項



繼承與權限控制

- 對於目前的瞭解，我們認為：
 - 繼承基本上就是把父類的成員與方法「挪」下來使用
 - 透過繼承，子類沒有辦法直接存取 `private` 的東西，但可以存取 `public` 與 `protected` 的東西

```
1  //
2  // Created by 黃漢軒 on 2023/10/9.
3  //
4
5  #ifndef OOP_CHARACTER_HPP
6  #define OOP_CHARACTER_HPP
7
8  #include <string>
9
10 class Character {
11 private:
12     std::string name;
13     int health;
14 public:
15     Character(std::string name, int health);
16
17     std::string GetName();
18     int GetHealth();
19
20     void SetName(std::string name);
21     void SetHealth(int health);
22 protected:
23     void SkillUsageCount();
24 };
25
26 #endif // OOP_CHARACTER_HPP
27
```


繼承與權限控制

- 所以，在 Ahri 的類別中，我可以直接存取 Character 的 GetHealth()
- 真的嗎？

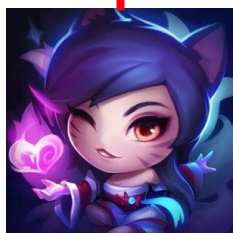
```
Ahri::Ahri(int abilityPower, std::string name, int health) : Mage(abilityPower, name, health) {  
    GetHealth();  
}  
void mage.h(3, 14): Constrained by implicitly private inheritance here  
character.h(13, 9): Member is declared here  
instance-method GetHealth  
void → int  
// In Character  
public: int GetHealth()  
void View Problem (Alt+F8) No quick fixes available
```

- GetHealth() 明明是 public，為什麼繼承下來變 private 了？

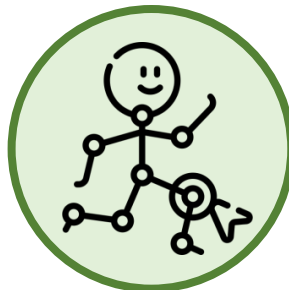
```
1 //  
2 // Created by 黃漢軒 on 2023/10/9.  
3 //  
4  
5 #ifndef OOP_CHARACTER_HPP  
6 #define OOP_CHARACTER_HPP  
7  
8 #include <string>  
9  
10 class Character {  
11     private:  
12         std::string name;  
13         int health;  
14     public:  
15         Character(std::string name, int health);  
16  
17         std::string GetName();  
18         int GetHealth();  
19  
20         void SetName(std::string name);  
21         void SetHealth(int health);  
22     protected:  
23         void SkillUsageCount();  
24 };  
25  
26 #endif // OOP_CHARACTER_HPP  
27
```


繼承與權限控制

- 對於 C++，我們有繼承與權限控制，分為：
 - Private inheritance
 - Public inheritance
 - Protected inheritance
- 當 Ahri 繼承 Mage 時，**預設為 private inheritance**
 - 所有 Mage 的 public 與 protected 成員與類別被視為 private
 - 因此，我們沒有辦法在外部呼叫阿璃的 GetAbilityPower()，因為它是 private 的



阿璃 Ahri



```
Ahri.hpp

1  //
2  // Created by 黃漢軒 on 2023/10/9.
3  //
4
5  #ifndef OOP_AHRI_HPP
6  #define OOP_AHRI_HPP
7
8  #include "mage.hpp"
9
10 class Ahri : Mage {
11
12     Ahri(std::string name, int health, int abilityPower);
13
14     void OrbOfDeception();
15     void FoxFire();
16     void Charm();
17     void SpiritRush();
18 };
19
20 #endif // OOP_AHRI_HPP
21
```

```
Mage.hpp

1  #ifndef OOP_MAGE_HPP
2  #define OOP_MAGE_HPP
3
4  #include "character.hpp"
5
6  class Mage : public Character {
7  private:
8      int abilityPower;
9
10 public:
11     Mage(std::string name, int health, int abilityPower);
12     int GetAbilityPower();
13     void SetAbilityPower(int newAp);
14     void WizardLevelUp();
15 };
16
17 #endif
```

繼承與權限控制

- 我們可以做 public inheritance
 - 基本上將 Mage 的 public 與 protected 函數繼承下來 (保持 public 或 protected)
 - private 依然保持 private

```
1 //
2 // Created by 黃漢軒 on 2023/10/9.
3 //
4
5 #ifndef OOP_AHRI_HPP
6 #define OOP_AHRI_HPP
7
8 #include "mage.hpp"
9
10 class Ahri : public Mage {
11
12     Ahri(std::string name, int health, int abilityPower);
13
14     void OrbOfDeception();
15     void FoxFire();
16     void Charm();
17     void SpiritRush();
18 };
19
20 #endif // OOP_AHRI_HPP
21
```

```
1 //
2 // Created by 黃漢軒 on 2023/10/9.
3 //
4
5 #ifndef OOP_MAGE_HPP
6 #define OOP_MAGE_HPP
7
8 #include "character.hpp"
9
10 class Mage : public Character {
11 private:
12     int abilityPower;
13 public:
14     Mage(std::string name, int health, int abilityPower);
15
16     int GetAbilityPower();
17     void SetAbilityPower(int abilityPower);
18     void WizardLevelUp();
19 };
20
21 #endif // OOP_MAGE_HPP
22
```

繼承與權限控制

- 透過下圖的方式，GetHealth() 對於 Mage 而言是 Public 的。
- 對於 Ahri 而言也是 Public 的，故 Ahri 可以直接使用 GetHealth()。

```
1 //
2 // Created by 黃漢軒 on 2023/10/9.
3 //
4
5 #ifndef OOP_AHRI_HPP
6 #define OOP_AHRI_HPP
7
8 #include "mage.hpp"
9
10 class Ahri : public Mage {
11
12     Ahri(std::string name, int health, int abilityPower);
13
14     void OrbOfDeception();
15     void FoxFire();
16     void Charm();
17     void SpiritRush();
18 }; GetHealth();
19
20 #endif // OOP_AHRI_HPP
21
```

```
1 //
2 // Created by 黃漢軒 on 2023/10/9.
3 //
4
5 #ifndef OOP_MAGE_HPP
6 #define OOP_MAGE_HPP
7
8 #include "character.hpp"
9
10 class Mage : public Character {
11 private:
12     int abilityPower;
13 public:
14     Mage(std::string name, int health, int abilityPower);
15
16     int GetAbilityPower();
17     void SetAbilityPower(int abilityPower);
18     void WizardLevelUp();
19 };
20
21 #endif // OOP_MAGE_HPP
22
```

```
1 #ifndef OOP_CHARACTER_HPP
2 #define OOP_CHARACTER_HPP
3
4 #include <string>
5
6 class Character {
7 private:
8     std::string name;
9     int health;
10
11 public:
12     Character(std::string name, int health);
13     std::string GetName();
14     int GetHealth();
15     void SetName(std::string newName);
16     void SetHealth(int newHealth);
17
18 protected:
19     void SkillUsageCount();
20 };
21
22 #endif
```


繼承與權限控制

- 對於三種不同的繼承權限控制，我們可以給出以下的表格。

		衍生類別		
		Public	Protected	Private
基礎類別	Public	Public	Protected	Private
	Protected	Protected	Protected	Private
	Private	Hidden	Hidden	Hidden

最終繼承：Final

- 最後，我們將描述 C++ 如何避免類別被繼承，以達到安全等相關的效益。



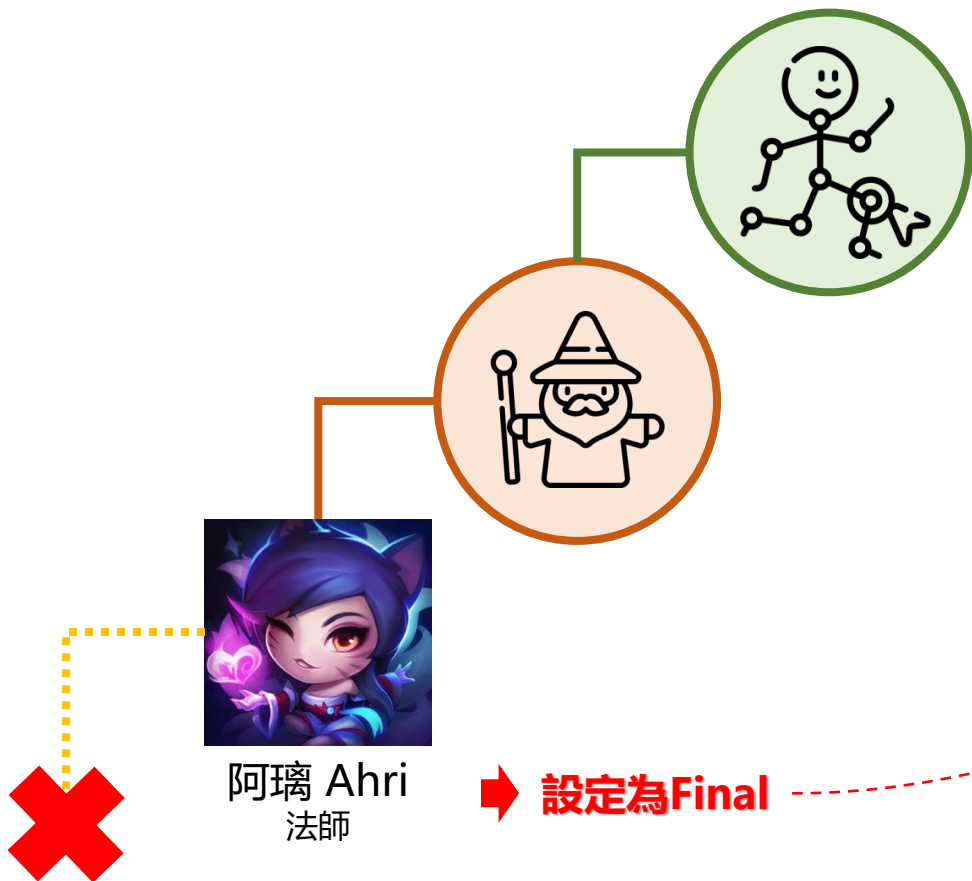
最終繼承：Final

- 任何東西都可以被繼承嗎？
 - 我們可以繼承 Character 這個類別，可以繼承 Mage 這個類別，但遊戲人物是能夠被繼承的嗎？
 - 以遊戲角色而言不應該被繼承，因為我們不會有多個阿璃，或者阿璃底下衍伸其他角色的問題
 - 基於嚴謹性與安全性，我們應該要防止該類別被繼承，並且在繼承時擋下
 - 這個時候，我們可以使用 Final



最終繼承：Final

- 透過 Final，我們可以宣稱該類別無法被繼承



```
1  //
2  // Created by 黃漢軒 on 2023/10/9.
3  //
4
5  #ifndef OOP_AHRI_HPP
6  #define OOP_AHRI_HPP
7
8  #include "mage.hpp"
9
10 class Ahri final : public Mage {
11
12     Ahri(std::string name, int health, int abilityPower);
13
14     void OrbOfDeception();
15     void FoxFire();
16     void Charm();
17     void SpiritRush();
18 };
19
20 #endif // OOP_AHRI_HPP
21
```

最終繼承：Final

- 一但有其他類別繼承了 Ahri, 就會出現編譯錯誤

```
//
#ifndef OOP_ZOE_HPP
#define OOP_ZOE_HPP

#include "ahri.hpp"
#include "mage.hpp"

class Zoe : public Ahri {
    void MoreSparkles();
    void PaddleStar();
    void SpellThief();
    void SpeelyTroubleBubble();
};

#endif // OOP_ZOE_HPP
```

- 透過這樣的方式，我們可以適時保護類別，使類別不該被繼承時就不該被繼承

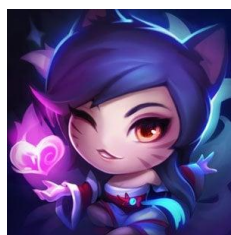
子類：覆寫函數

- 能不能滿足「少數類別」需要
改寫繼承來的函數？

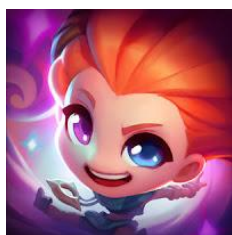


子類：覆寫函數

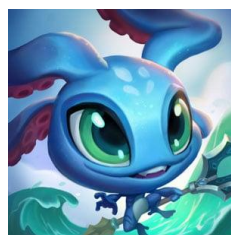
- 對於現有繼承架構來說，他大幅度的抽出了許多共同成員與函數



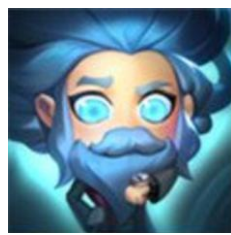
阿璃 Ahri
法師



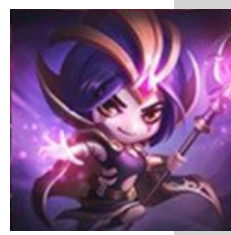
柔伊 Zoe
法師



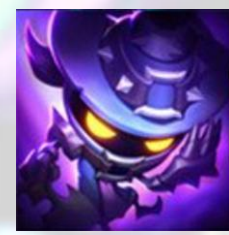
飛斯 Fizz
法師



XXX
法師



YYY
法師



ZZZ
法師

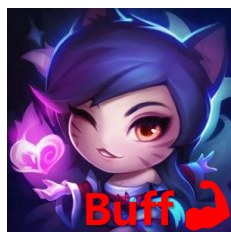
共同 $Level \times 0.5 \times AbilityPower$



WizardLevelUp()

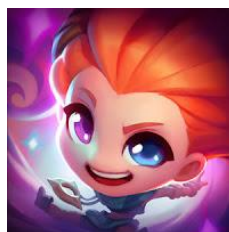
子類：覆寫函數

- 但我們可能要開始面對一些有趣的問題
- 「如果今天只想要讓阿璃有自己的 WizardLevelUp 特性呢」

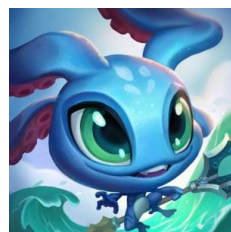


阿璃 Ahri
法師

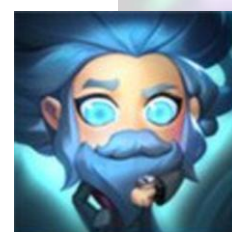
$$\text{Level} \times 1.5 \times \text{AbilityPower}$$



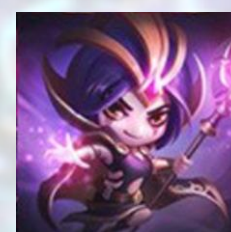
柔伊 Zoe
法師



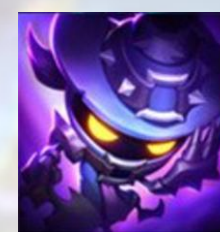
飛斯 Fizz
法師



XXX
法師



YYY
法師



ZZZ
法師

$$\text{Level} \times 0.5 \times \text{AbilityPower}$$

- 如果遇到萬中選一的特判，要怎麼保持程式的整潔度呢？

子類：覆寫函數

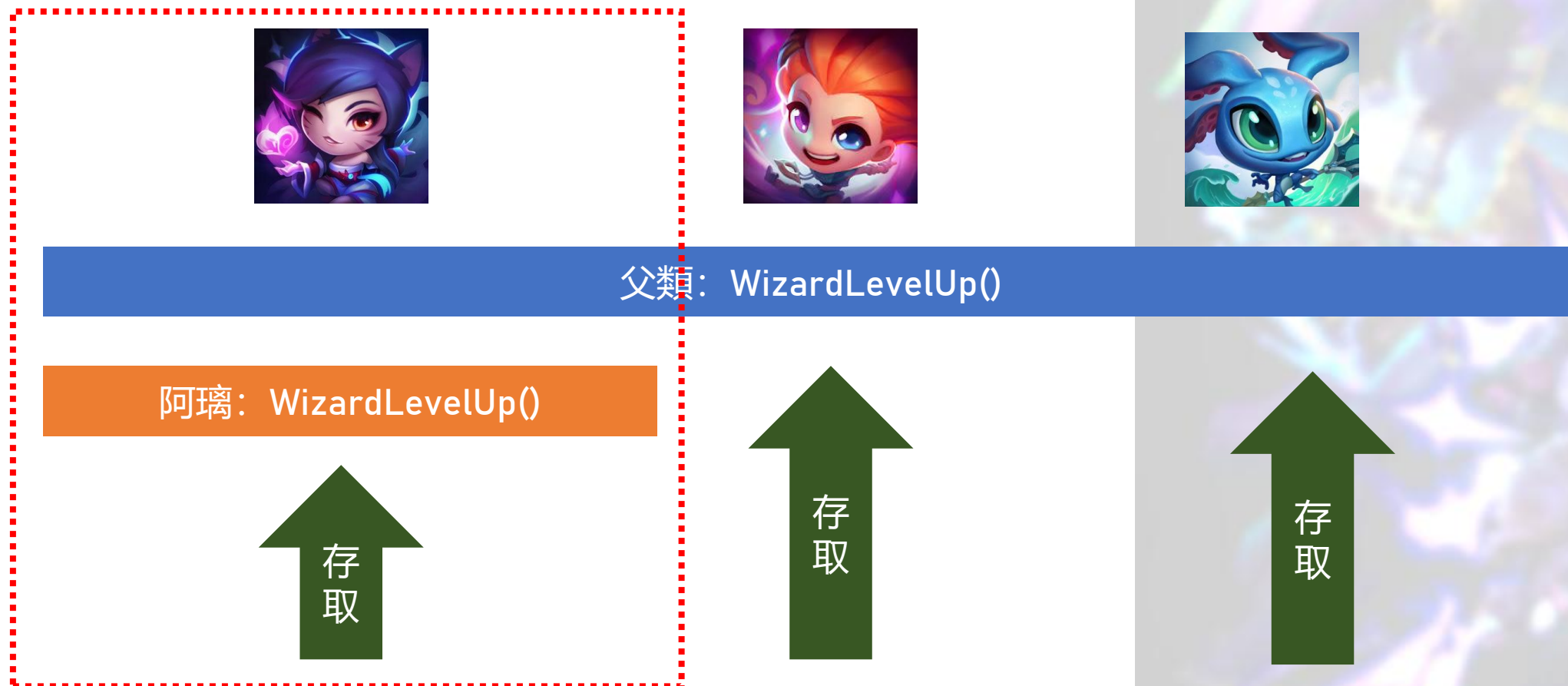
- 議題回顧：
 - 身為一個很棒的 LoL 開發者，你希望可以做一個專屬於阿璃的 WizardLevelUp 函數
 - 當 WizardLevelUp 喪失了共同性，所以必須要把所有的遊戲角色類別的 WizardLevelUp 全部還原回去
 - Good/Bad Practice?
 - 聽起來超不合理，如果遊戲中有 100 個法師角色，我就得要寫 99 個 WizardLevelUp 函數，這樣的可維護性變得超級糟
- 動動腦，假設是你，你會希望怎麼樣的方式可以解決掉這個議題呢？

子類：覆寫函數

- 覆寫（Overriding）是物件導向程式設計中的一個概念，出現在有繼承（Inheritance）關係的類別中
- 當子類別繼承了父類別的某個方法後，若子類別希望該方法有不同的行為，那麼子類別可以重新定義該方法，這種行為被稱為"覆寫"該方法
- 這時候我們可以針對於「父類」進行函數的覆寫

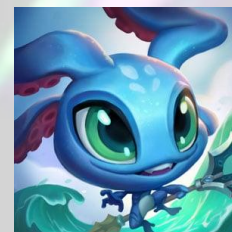
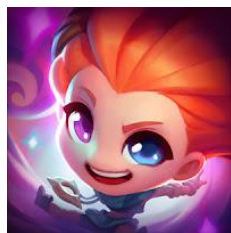
子類：覆寫函數

- 函數的覆寫主要概念是「我覆寫了這個 function，所以我這個類別呼叫這個 function 時，應該要用我的 function」



子類：覆寫函數

- 當該類別沒有特別進行覆寫，則他會使用父類所定義的 WizardLevelUp() 函數



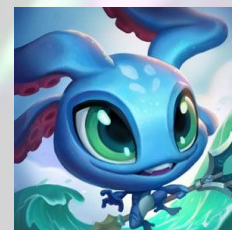
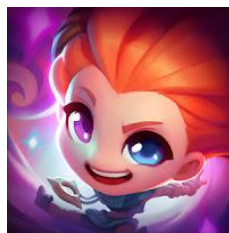
父類: WizardLevelUp()

阿璃: WizardLevelUp()



子類：覆寫函數

- 所以也當然的，當如果不僅一個類別需要自定義 WizardLevelUp() 時，我們也可以幫它特製一套



父類: WizardLevelUp()

阿璃: WizardLevelUp()

存取

存取

飛斯: WizardLevelUp()

存取

子類：覆寫函數

- 透過這樣的方式，我們可以在保持繼承所帶來的好處時，能夠針對類別進行特設，大幅增加了靈活性。

父類： 虛擬函數

- 在這個章節中，我們將繼續嘗試描述如何使用繼承的特性，將特定函數指派給底下的子類。



父類： 虛擬函數

- 如果 C++ 的函數可以被覆寫的話：
 - 我可以覆寫任意一個 function 嗎？
 - 什麼樣的函數可以被覆寫呢？
 - 我要怎麼防止一個 function 被覆寫呢？

父類： 虛擬函數

- 在 C++ 中，只有虛擬函數 (Virtual function) 能夠被覆寫
- 當一個函數被宣告為虛擬函數時，子類能夠選擇：
 - 是否沿用父類所定義的函數
 - 或者嘗試覆寫父類所定義的函數

```
1  #include "character.h"
2
3  class Mage : Character {
4  private:
5      int abilityPower;
6  public:
7      Mage(int abilityPower, std::string name, int health);
8
9      /* Getter */
10     int GetAbilityPower();
11
12     /* Setter */
13     void SetAbilityPower(int abilityPower);
14
15     /* Wizard-only */
16     virtual void WizardLevelUp();
17 };
```

父類： 虛擬函數

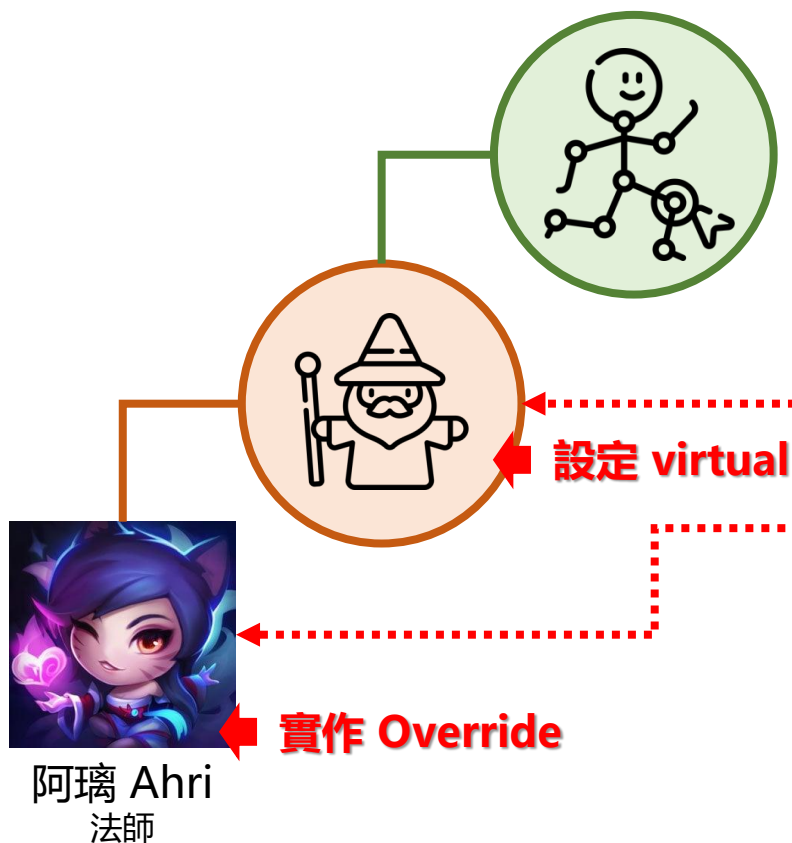
- 例如我們想要讓 WizardLevelUp() 函數能夠被覆寫，則需要滿足以下的事項：
 - 父類 WizardLevelUp() 函數必須要是 Virtual
 - 子類需要有一個 WizardLevelUp() 的 Virtual 函數，並且加上 override 來覆寫它

```
1  #include "character.h"
2
3  class Mage : Character {
4  private:
5      int abilityPower;
6  public:
7      Mage(int abilityPower, std::string name, int health);
8
9      /* Getter */
10     int GetAbilityPower();
11
12     /* Setter */
13     void SetAbilityPower(int abilityPower);
14
15     /* Wizard-only */
16     virtual void WizardLevelUp();
17 };
```

```
1  #include <string>
2
3  #include "mage.h"
4
5  class Ahri : Mage {
6  public:
7      Ahri(int abilityPower, std::string name, int health);
8      /* Ahri-Only Skill */
9      void OrbOfDescription();
10     void FoxFire();
11     void Charm();
12     void SpiritRush();
13
14     virtual void WizardLevelUp() override; // 然後定義自己的 WizardLevelUp()
15 };
```


父類： 虛擬函數

- 示意圖



```
1  #include "character.h"
2
3  class Mage : Character {
4  private:
5      int abilityPower;
6  public:
7      Mage(int abilityPower, std::string name, int health);
8
9      /* Getter */
10     int GetAbilityPower();
11
12     /* Setter */
13     void SetAbilityPower(int abilityPower);
14
15     /* Wizard-only */
16     virtual void WizardLevelUp();
17 };
```

```
1  #include <string>
2
3  #include "mage.h"
4
5  class Ahri : Mage {
6  public:
7      Ahri(int abilityPower, std::string name, int health);
8      /* Ahri-Only Skill */
9      void OrbOfDescription();
10     void FoxFire();
11     void Charm();
12     void SpiritRush();
13
14     virtual void WizardLevelUp() override; // 然後定義自己的 WizardLevelUp()
15 };
```

父類： 虛擬函數

- 透過這樣的方式，當我們存取 Ahri 的 WizardLevelUp() 函數時，則我們存取到的會是 Ahri 覆寫的 WizardLevelUp() 函數。



父類： 虛擬函數

- 其他的類別（柔伊、飛斯）將保持原本父類定義的 WizardLevelUp() 虛擬函數。



父類： 虛擬函數

- 透過這樣的方式，我們可以定義虛擬函數，使子類可以覆寫虛擬函數，或者維持父類所定義的虛擬函數
- 透過虛擬函數，使我們的類別更加靈活，可以針對於需求進行覆寫，或者沿用父類的定義

純虛擬函數 vs 虛擬函數

- 能否先不要實作細節，將特定的函數交給子類別來處理？



純虛擬函數與虛擬函數

- 在「純虛擬函數與虛擬函數」這個章節中，我們將嘗試解決第二個議題。
 - 「如果我今天想要讓阿璃有自己的 WizardLevelUp 特性呢」 - 虛擬函數
 - 「如果我今天想要**強制**讓所有的法師有自己的 WizardLevelUp 特性呢」



我們法師要有WizardLevelUp()
請各組設計自己的！

工程師忘記實作...(報錯誤訊息!)

- 對於現有架構來說，我們想要讓法師有自己獨特的 WizardLevelUp 的特性
 - 就期望上，我們要求每個法師必須要自己實作 WizardLevelUp
 - 那假設在設計上忘記實踐，或者沒有實踐的話，要怎麼即時知道錯誤呢？

純虛擬函數與虛擬函數

- 對於虛擬函數來說，分為「純虛擬函數 Pure virtual function」與「虛擬函數 Virtual Function」
 - 虛擬函數**不強制**一定要 override，可以採用父類的定義或子類 override 成自己的定義
 - 純虛擬函數**強制**一定要 override，父類的定義不應該存在，子類一定要自己定義
 - 如果再父類別上使用pure virtual function，但又再父類別的.cpp上實作方法，會被編譯器阻擋
- 因此在概念上，比較像是父類交由子類別自己定義該 Function，並且自己使用

純虛擬函數與虛擬函數

- 在純虛擬函數中，我們需要將虛擬函數設置為「=0」來代表該函數是純虛擬函數
 - Mage無法定義WizardLevelUp
 - 誰繼承Mage就要自己完成定義

```
1  #include "character.h"
2
3  class Mage : Character {
4  private:
5      int abilityPower;
6  public:
7      Mage(int abilityPower, std::string name, int health);
8
9      /* Getter */
10     int GetAbilityPower();
11
12     /* Setter */
13     void SetAbilityPower(int abilityPower);
14
15     /* Wizard-only */
16     virtual void WizardLevelUp() = 0;
17 };
```

純虛擬函數與虛擬函數

- 當子類進行繼承後，必須要實作該虛擬函數，否則在運行上會出現錯誤
- 因為父類別並沒有實作該 Function，而是強制將該實作交由子類



純虛擬函數與虛擬函數

- 因此，純虛擬函數可用於交付設計概念給子類別，使子類別自己設計自己的 Function
- 透過這樣的方式，我們可以硬性規定繼承的子類別必須要實作該 Function，使子類別應實作屬於自己的 Function